



QDMS Summer School
July 18, 2025

Molecular Dynamics, Density Functional Theory, and QM/MM on GPUs

Andreas W Götz

San Diego Supercomputer Center
School of Computing, Information and Data Sciences
University of California, San Diego

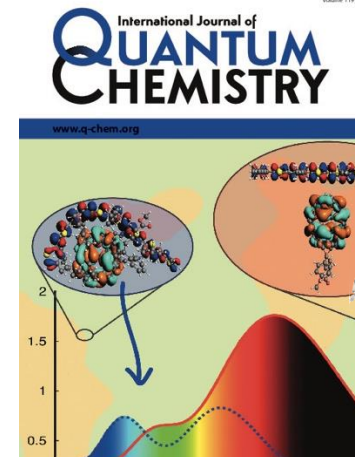
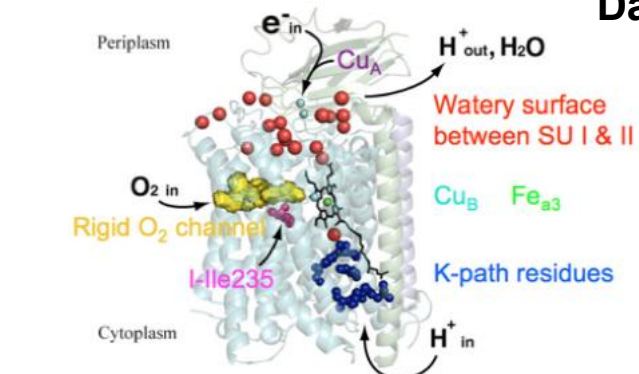
Research Interests

Molecular simulations

Quantum mechanics, Molecular Dynamics

Method and software development

Data-driven science, High-Performance Computing



```
! This subroutine computes the MM character (lambda) of
! a solvent molecule in the T-region
subroutine calculate_lambda (dist, RA, RT, lambda)
```

```
use constants, only : two, three
implicit none
```

```
!Passed in
REAL, intent(in) :: dist, RA, RT
REAL, intent(out) :: lambda
```

```
if (dist < RA) then
  lambda = 0.00
```

```
else if ((dist >= RA) .and. (dist
  lambda = ((dist - RA)**two) *
  lambda = 1.00
```

```
end if
```

```
type nam_adaptive_type, intent(i
  intent(i
  intent(i
```

```
! Local
integer :: ist(MPI_STATU
```

```
!bug then
  '(a)' '>>>>>
  ish(6)
```

```
ind(force, 3*ni
  0, 0, comm
```

```
return
```

```
end
```

```
end
```

```
end
```

```
end
```

```
end
```

```
end
```

```
end
```

```
end
```

```
end
```

```
end
```

```
end
```

```
end
```

```
end
```

```
end
```

```
end
```

```
end
```

```
end
```

```
end
```

```
end
```

```
end
```

```
end
```

```
end
```

```
end
```

```
end
```

```
end
```

```
end
```

```
end
```

```
end
```

```
end
```

```
end
```

Journal of
**COMPUTATIONAL
CHEMISTRY**

Organic • Inorganic • Physical
Biological • Materials

www.c-chem.org

www.c-chem.org

www.c-chem.org

www.c-chem.org

www.c-chem.org

www.c-chem.org

www.c-chem.org

www.c-chem.org

www.c-chem.org

www.c-chem.org

www.c-chem.org

www.c-chem.org

www.c-chem.org

www.c-chem.org

www.c-chem.org

www.c-chem.org

www.c-chem.org

www.c-chem.org

www.c-chem.org

www.c-chem.org

www.c-chem.org

www.c-chem.org

www.c-chem.org

www.c-chem.org

www.c-chem.org

www.c-chem.org

www.c-chem.org

www.c-chem.org

www.c-chem.org

www.c-chem.org

www.c-chem.org

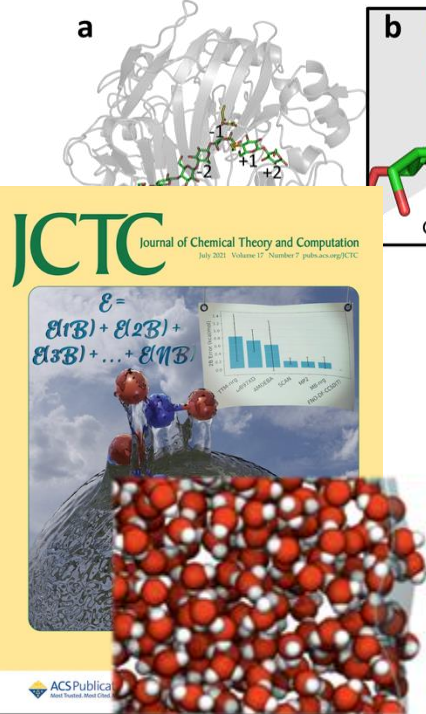
www.c-chem.org

www.c-chem.org

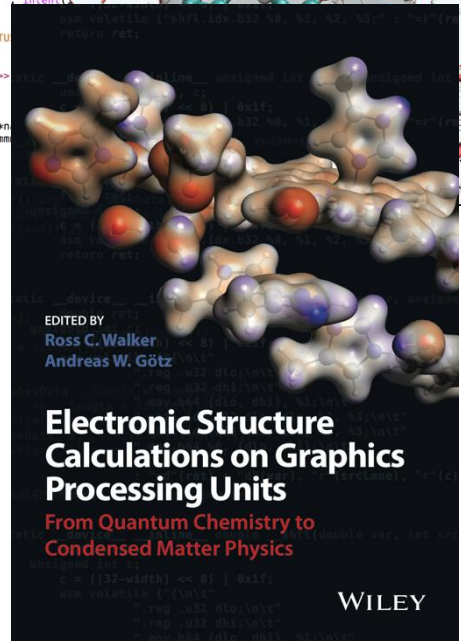
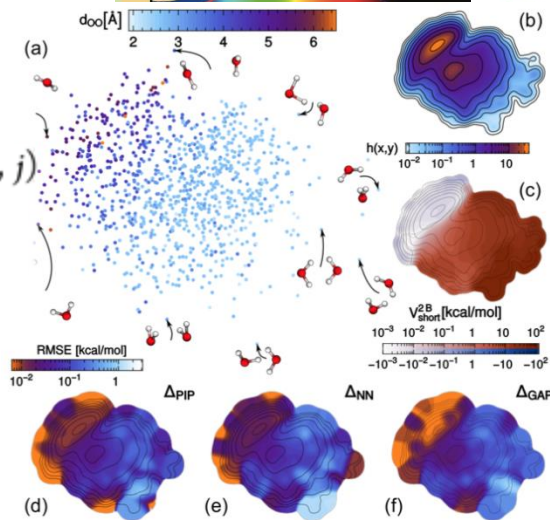
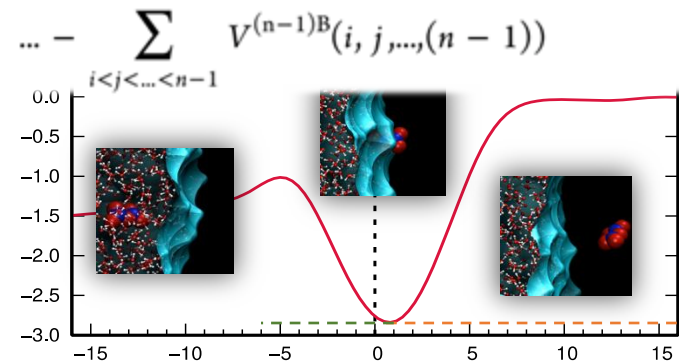
www.c-chem.org

www.c-chem.org

www.c-chem.org



$$V^{nB}(1, \dots, n) = E_n(1, \dots, n) - \sum_i V^{1B}(i) - \sum_{i < j} V^{2B}(i, j) - \dots - \sum_{i < j < \dots < n-1} V^{(n-1)B}(i, j, \dots, (n-1))$$



SDSC – 40 years of supercomputing

- **1985** Established as a national supercomputer resource center by the US National Science Foundation
- **1997** Integrated with UC system
- Now a department of UC San Diego in the School of Computing, Information and Data Science
- ~250 people, > 100 researchers
- Primarily funded through research grants, but SDSC also sells services



A cavalcade of supercomputers

1985 – Cray X-MP



1989 – Cray Y-MP



1993 – Cray C90



1996 – Cray T3E



architectural convergence



1990 – NCUBE 2



1993 – Intel Paragon



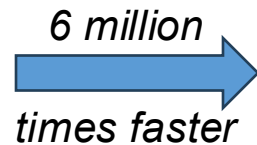
1994 – Thinking Machines
CM-2

SDSC supercomputers then and now

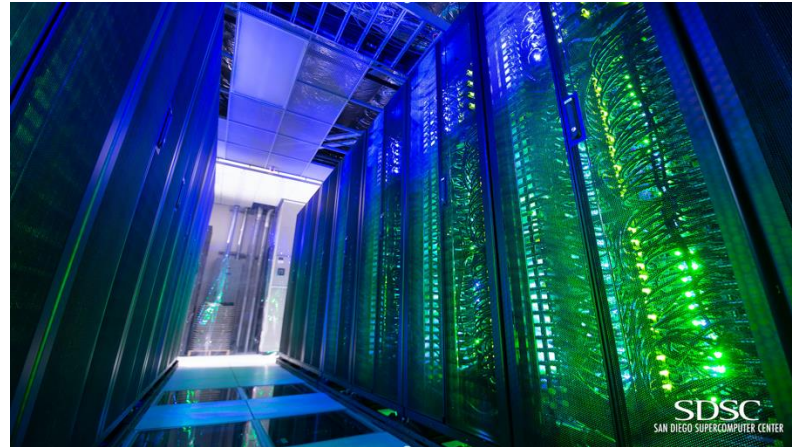
1985 – Cray X-MP



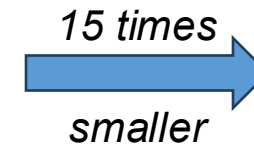
800 MFLOPS
4x Vector CPU



2020 – Expanse
Dell CPU + GPU nodes
Production system



5.2 PFLOPS (HPC)
AMD EPYC CPUs
Nvidia V100 GPUs
(+ Nvidia H100 GPUs
in AIR Expansion)



2025 – SDSC Cosmos
HPE Cray XE2500
Testbed system

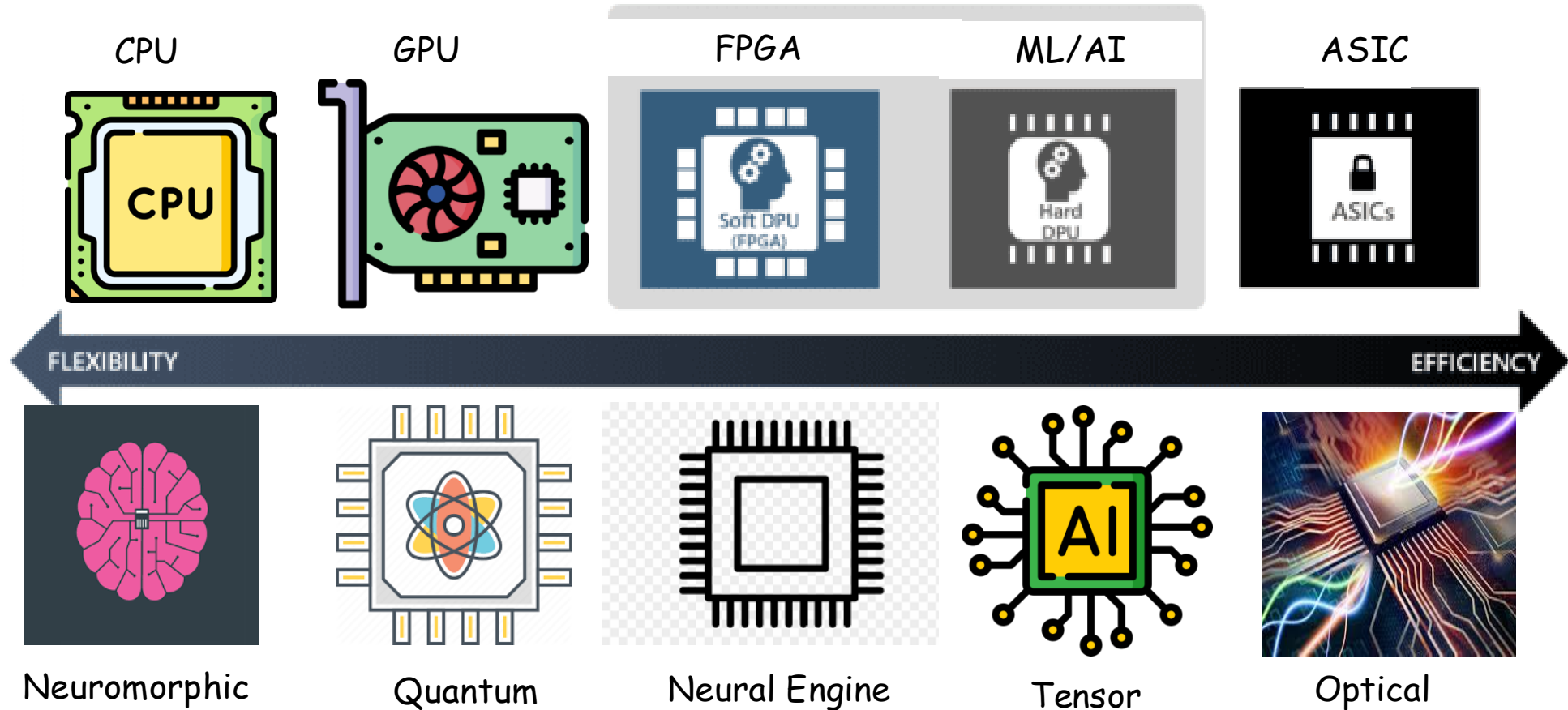


10 PFLOPS (HPC)
100 PFLOPS (AI)
AMD MI300A APUs

Future HPC Systems Will be Customized...

Specialized hardware

- Different types of accelerators for different types of applications / compute workloads
- Computational scientists will need to develop algorithms and software for this hardware



Microprocessor Trends

Moore's law

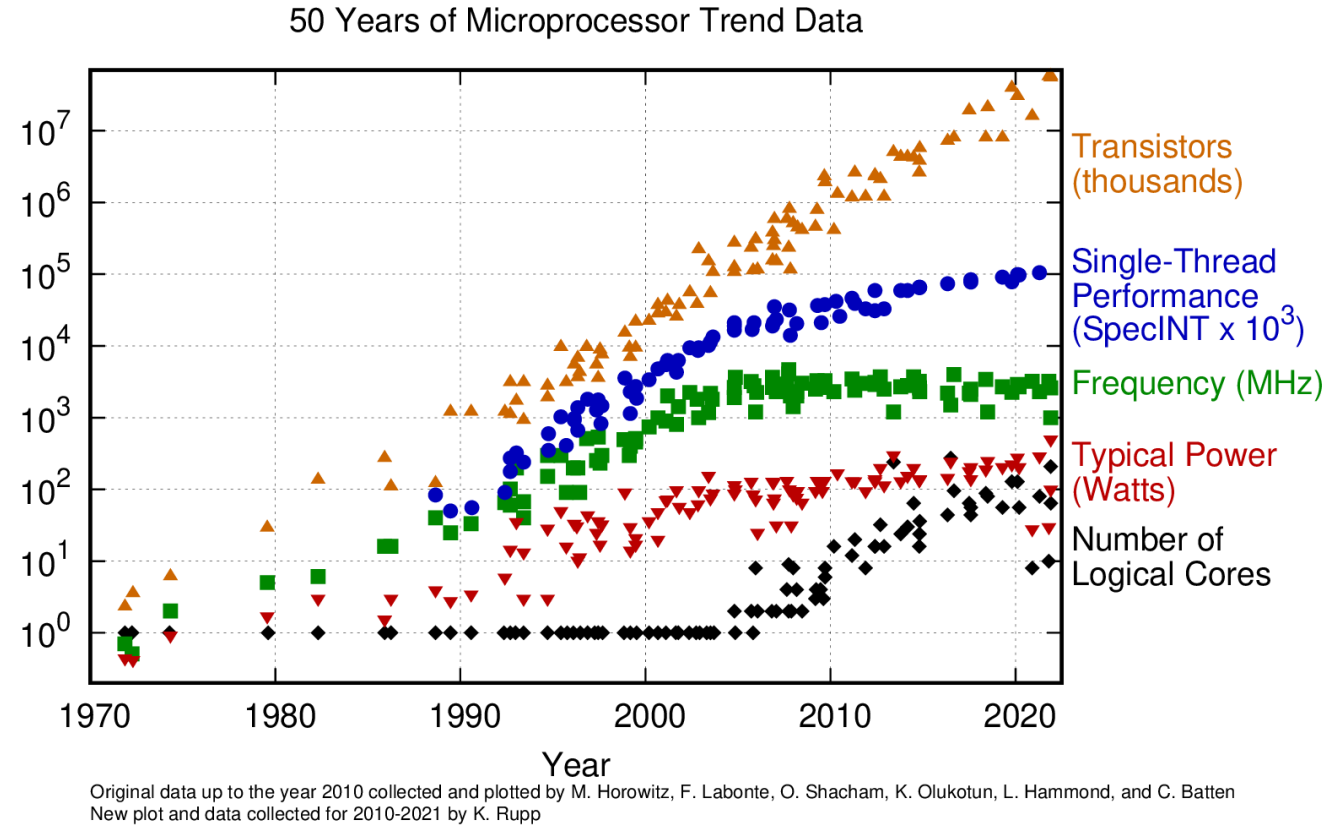
- Transistor count in integrated circuits doubles about every two years.
- Exponential growth still holds (see figure).
- However...

Trends since mid 2000s

- Clock frequency constant.
- Single CPU core performance (serial execution) roughly constant.
- Performance increase due to increase of CPU cores per processor.

Impact on software

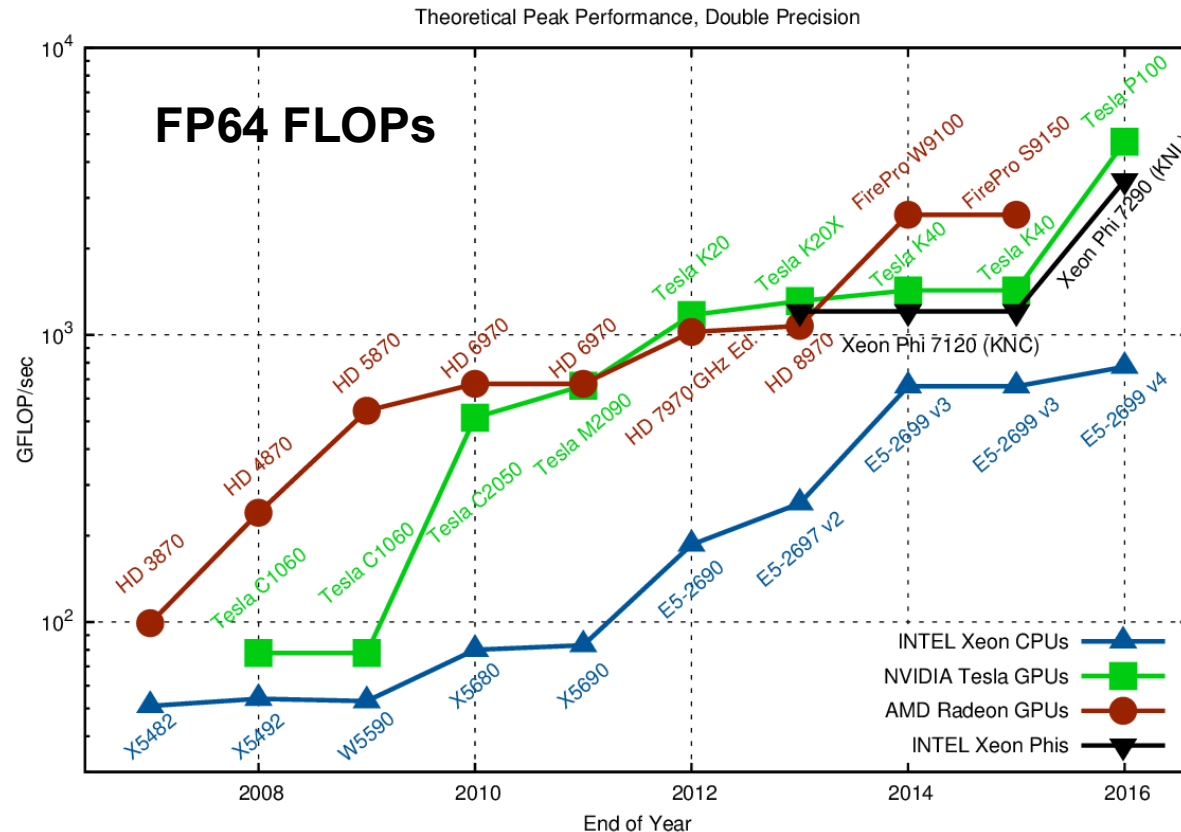
- Cannot simply wait two years to double code execution performance.
- Must write parallel code.



Source:

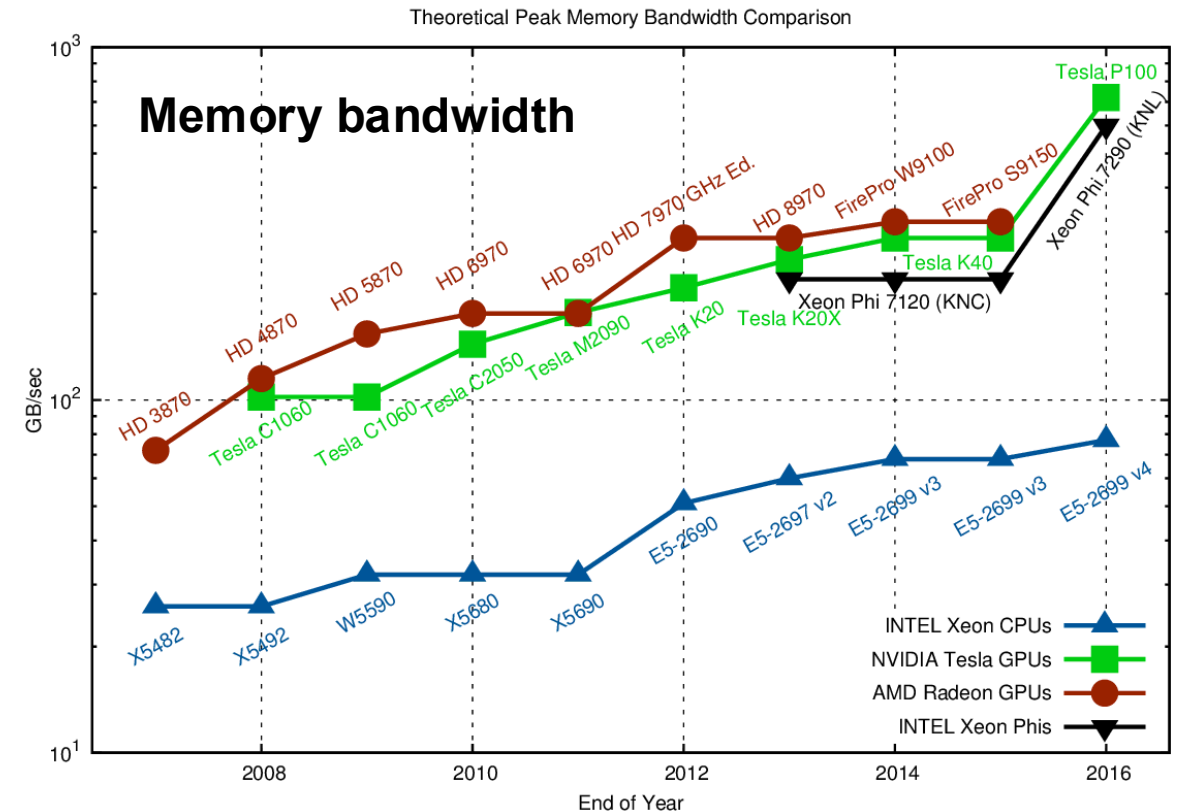
<https://github.com/karlrupp/microprocessor-trend-data/tree/master/50yrs>

Emergence of GPUs Over the Last 20 Years



Development driven by

- O(\$100 billion) gaming industry
- O(\$100 billion) AI industry, exponentially growing
- NSF budget is \$9 billion, NIH budget \$47 billion (FY24)



GPUs are good at

- Data parallel computations
- Matrix algebra
- Well suited for Deep Learning workloads

A supercomputer in a desktop?



ASCI White (LLNL)

- **12.3 TFLOP/sec** – #1 Top 500, November 2001.
- Cost – \$110 Million USD (in 2001!)



SDSC Expanse

- 728 CPU nodes with 4.6 TFLOP/sec (each node)
3.4 PFLOP/sec (aggregate CPU)
- 52 GPU nodes 4 x Nvidia V100 (Volta arch)
31.3 TFLOP/sec FP64, 62.7 TFLOP/sec SP (each node)
1.6 PFLOP/sec FP64, 3.3 PFLOP/sec FP32 (aggregate GPU)
- Hardware Cost – \$10 Million USD



DIY 4 x Nvidia RTX 3080 box (2020) (Ampere arch)

- 1.9 TFLOP/sec FP64
- **119.0 TFLOP/sec FP32**
- Cost – ~\$4 Thousand USD

SDSC Expanse - launched in Fall 2020

EXPANSE

COMPUTING WITHOUT BOUNDARIES
5 PETAFLOP/S HPC and DATA RESOURCE

HPC RESOURCE

- 13 Scalable Compute Units
- 728 Standard Compute Nodes
- 52 GPU Nodes: 208 GPUs
- 4 Large Memory Nodes

LONG-TAIL SCIENCE

- Multi-Messenger Astronomy
- Genomics
- Earth Science
- Social Science

INNOVATIVE OPERATIONS

- Composable Systems
- High-Throughput Computing
- Science Gateways
- Interactive Computing
- Containerized Computing
- Cloud Bursting

DATA CENTRIC ARCHITECTURE

- 12PB Perf. Storage: 140GB/s, 200k IOPS
- Fast I/O Node-Local NVMe Storage
- 7PB Ceph Object Storage
- High-Performance R&E Networking

REMOTE CI INTEGRATION

CLOUD



Heterogeneous Resources

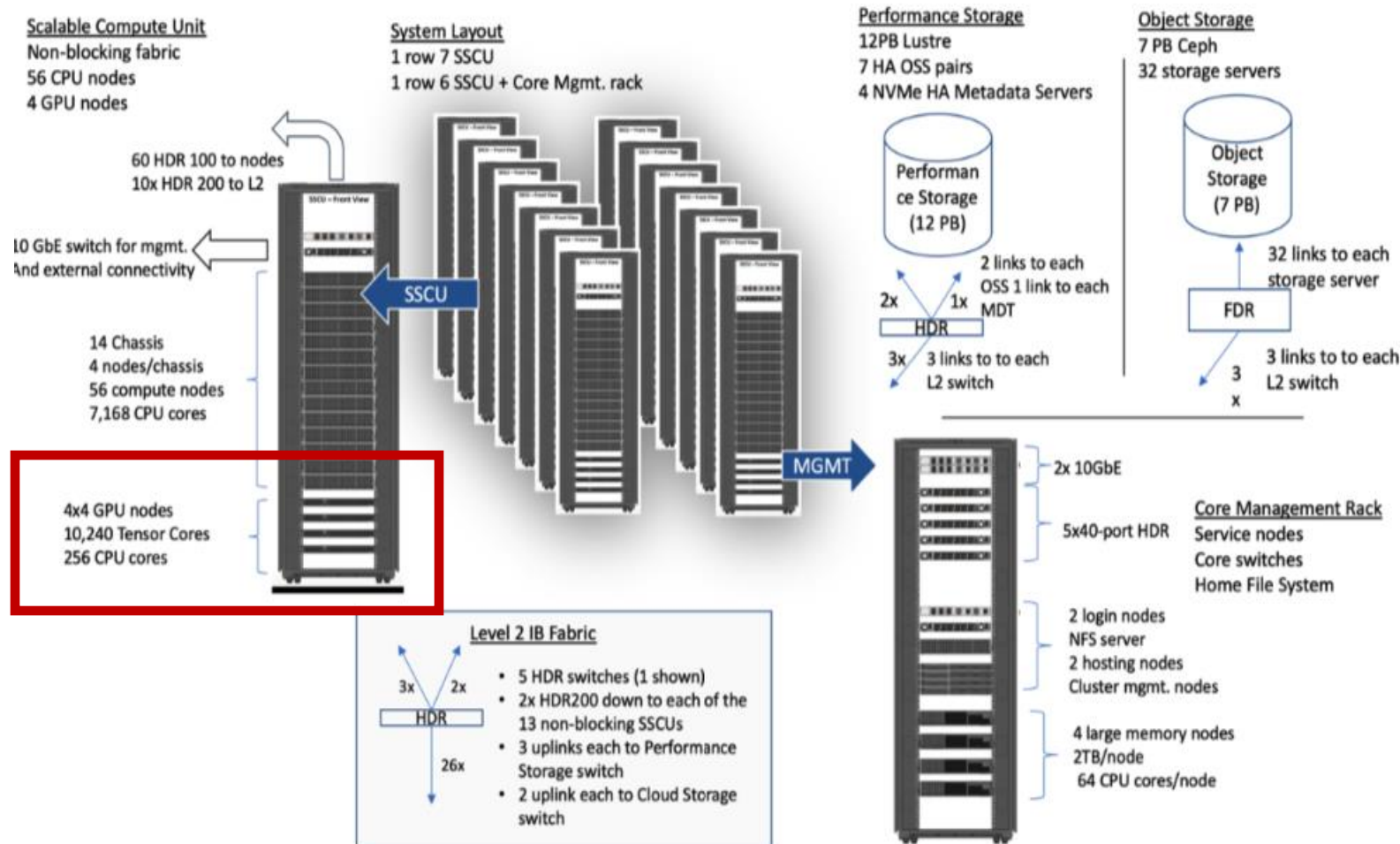


Open Science Grid

SDSC Expanse Heterogeneous Architecture

System Summary

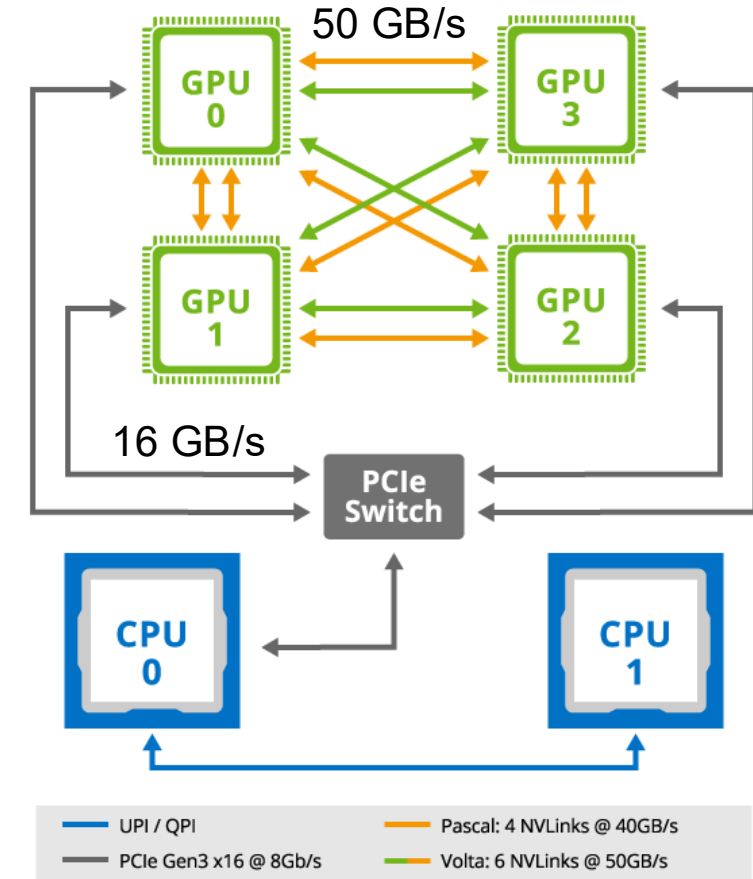
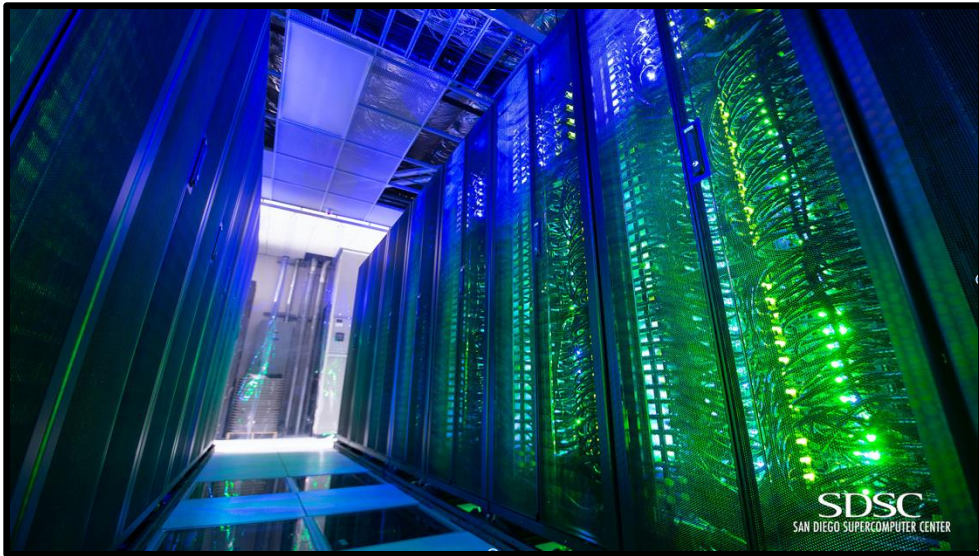
- 13 SDSC Scalable Compute Units (SSCU)
- 728 Standard Compute Nodes
- 93,184 Compute Cores
- 200 TB DDR4 Memory
- 52x 4-way GPU Nodes w/NVLink
- **208 V100 GPUs**
- 4x 2TB Large Memory Nodes
- HDR 100 non-blocking Fabric
- 12 PB Lustre High Performance Storage
- 7 PB Ceph Object Storage
- 1.2 PB on-node NVMe
- Dell EMC PowerEdge
- Direct Liquid Cooled



SDSC Expanse GPU Nodes with Nvidia V100 SXM2

52 GPU nodes

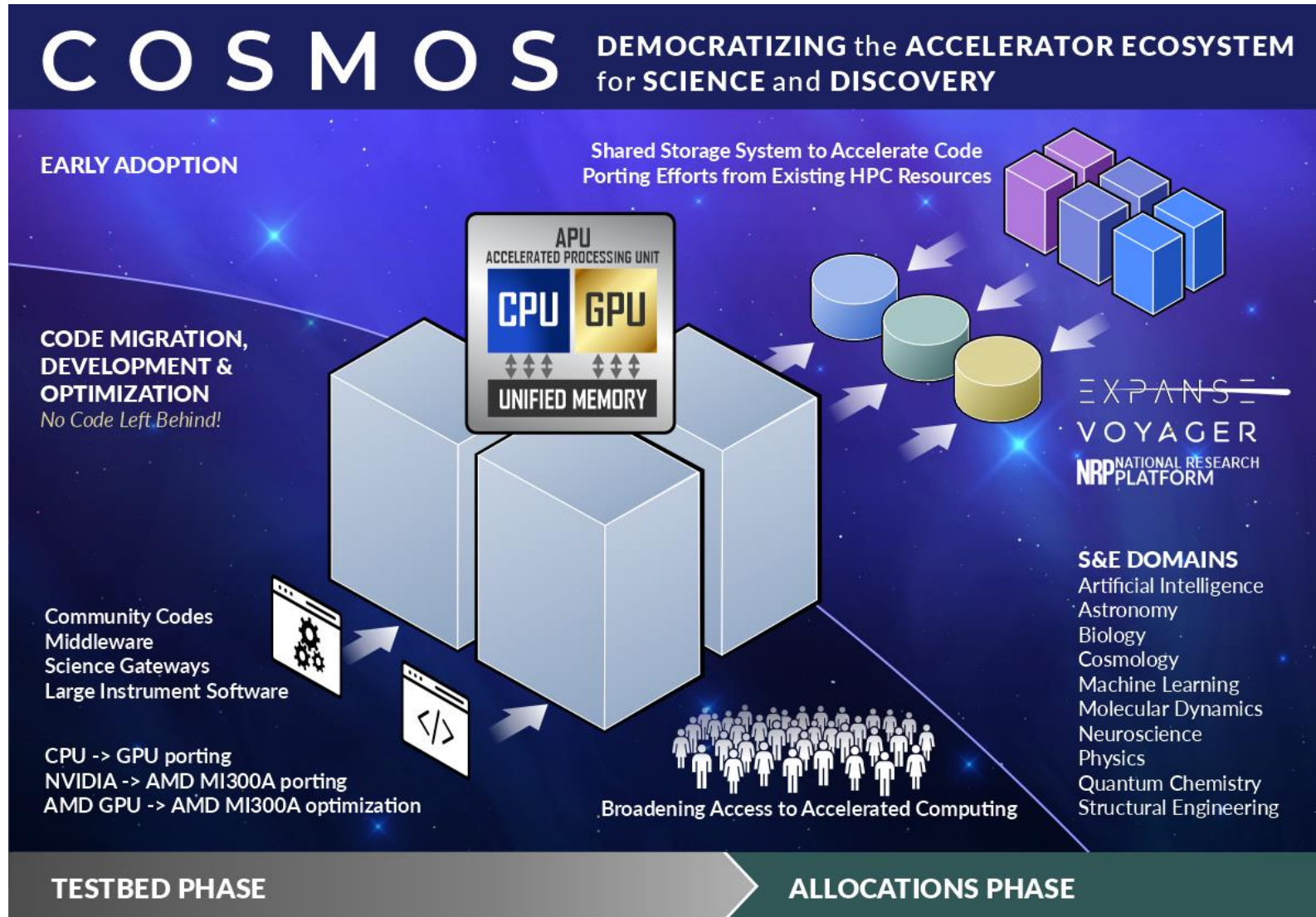
- 2 x 20-core Intel Xeon Gold 6248 (Cascade Lake) CPUs
- 384 GB RAM (131 GB/s)
- 4 x Nvidia V100 SXM2 GPUs
- 32 GB HBM2 RAM per GPU (897 GB/s)
- 1.6 TB NVMe/node



User guide:

https://www.sdsc.edu/support/user_guides/expance.html

SDSC Cosmos - launched in May 2025



Cosmos in SDSC data center

SDSC Cosmos - launched in May 2025

HPE CRAY EX2500 Supercomputer



Single rack of EX2500



Compute Chassis: Provides power, cooling, system control, and network fabric for up to 8 compute blade slots

3 Chassis per rack, 8 blades per chassis, each EX255a blade can support:

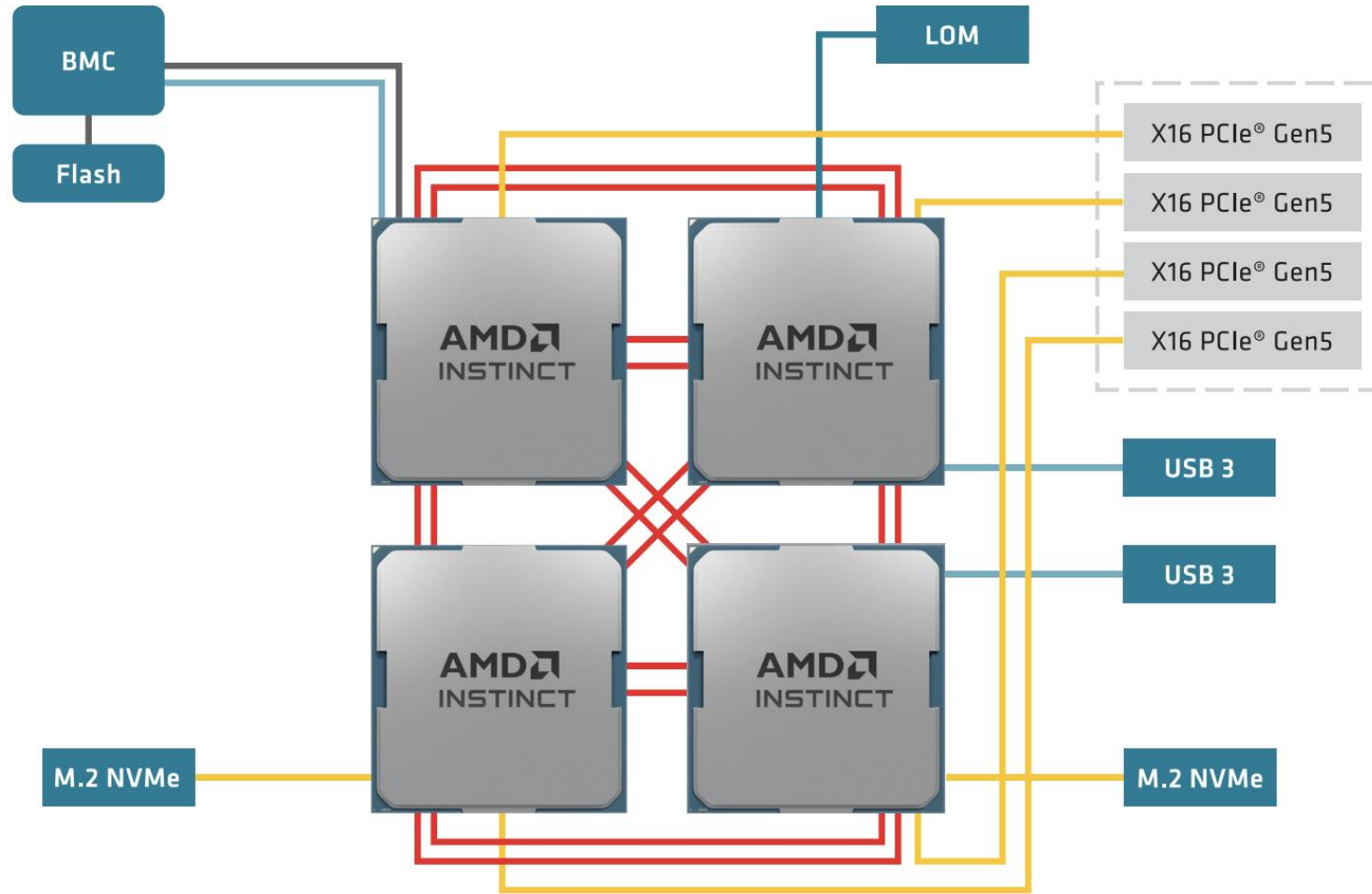
- Two 4-socket AMD MI300a Accelerator APU nodes
- 128GB HBM3 per APU
- Up to 8 HPE Slingshot 200Gbit/sec ports per blade
- 1 local NVMeM.2SSD per node (up to 2 per blade)
- 2 Board Management Controllers (BMC) per blade
- Cooled with cold plate

Overall system configuration

- 42 nodes, each with 4 APUs (168 total) integrated into full system using HPE's Slingshot technology
- High-performance storage from VAST (~300TB) that provides the high IOPS, and bandwidth needed for mixed-application workload
- 5 PB of Ceph capacity storage
- Qumulo storage for home filesystem
- HPE EX2500 rack with liquid cooling

Reference: <https://www.hpe.com/psnow/doc/a00094635enw>

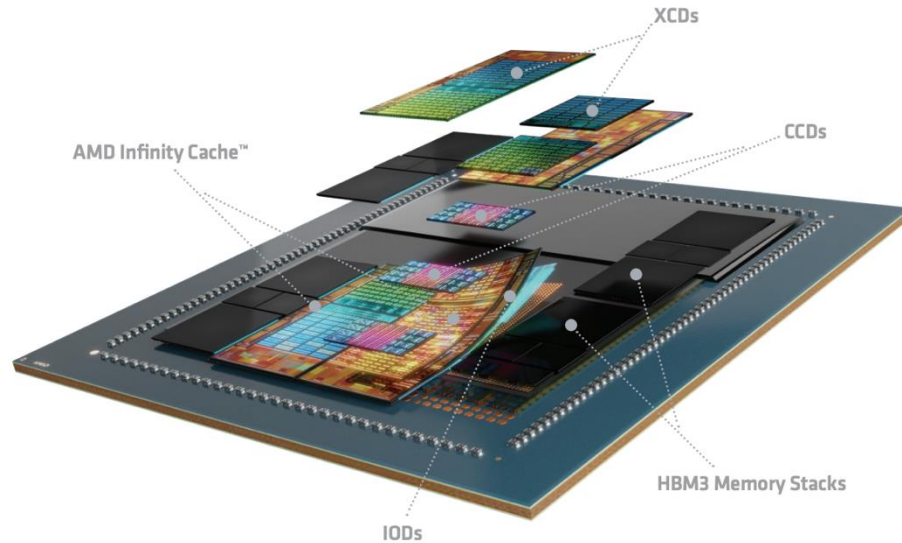
SDSC Cosmos – 4 APU's per node



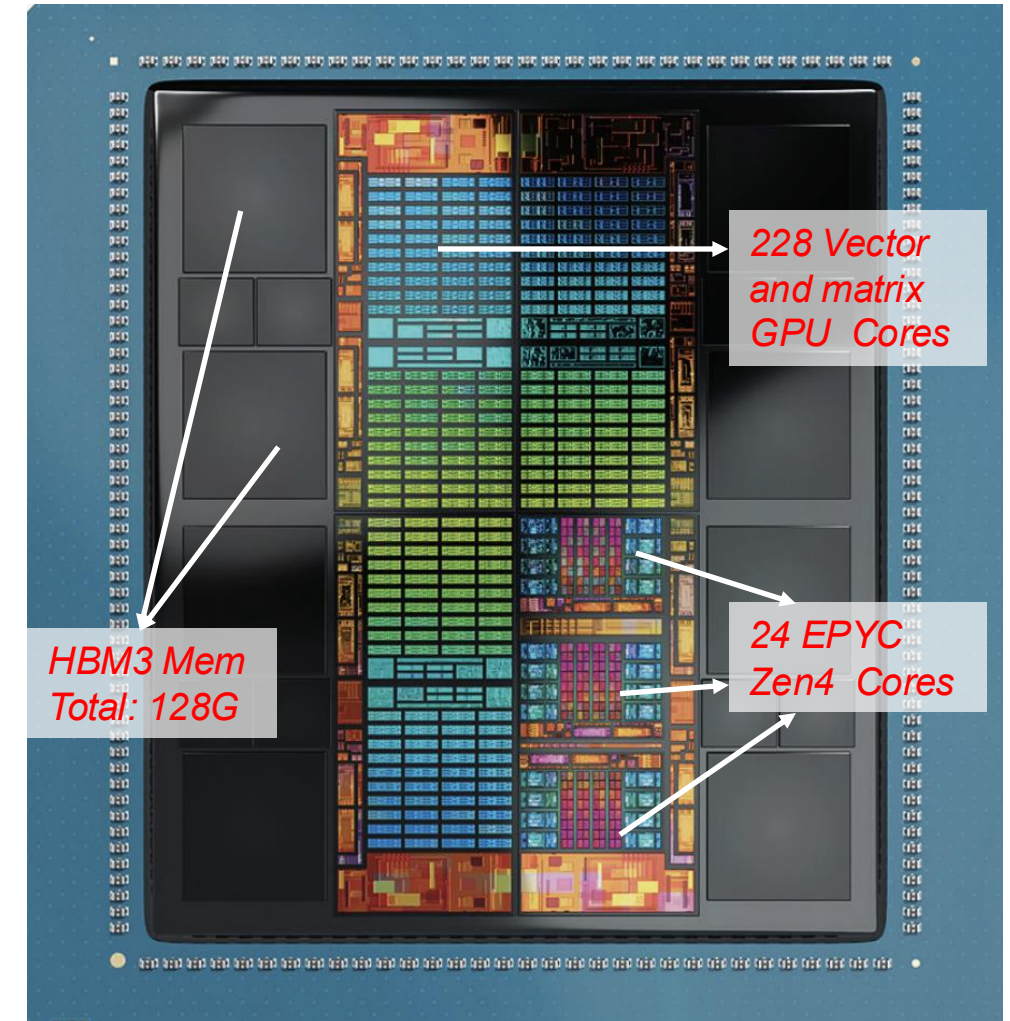
- 4 MI300A APU's fully connected via AMD's inter-chip global memory interface (xGMI)
- xGMI provides 768 GB/s aggregate bi-directional bandwidth among all the APU's, 256 GB/s bi-directional peer-to-peer bandwidth between each pair of APU's.
- Each APU is served by a 200-Gbps Slingshot NIC, which provides connectivity to the Slingshot interconnect

Reference: <https://www.amd.com/content/dam/amd/en/documents/instinct-tech-docs/white-papers/amd-cdna-3-white-paper.pdf>

SDSC Cosmos - AMD MI300A Architecture



- MI300A APU combines CPU, GPU, and memory on one package. Six accelerated compute dies (XCDs) combined with 3 chiplets with 8 zen4 cores each.
- 128 GB of HBM3 memory shared coherently between CPUs and GPUs. 5.3 TB/s peak throughput.
- 256 MB AMD Infinity Cache (last level) shared between XCDs and CPUs
- Design helps eliminate need to do host/device data copies and eases code development



Reference: <https://www.amd.com/content/dam/amd/en/documents/instinct-tech-docs/white-papers/amd-cdna-3-white-paper.pdf>

Current HPC Environment for Scientific Computing

DOE Exascale Supercomputers



AMD Based (APUs)



Intel Based (CPU + GPU)



AMD Based (CPU + GPU)

SDSC Expanse



AMD + Intel CPUs
Nvidia GPUs

SDSC Voyager



Intel CPU and
Habana AI processors

SDSC Cosmos



AMD Based (APUs)

Highly parallel

- Distributed memory
- MPI + Open-MP programming model

Heterogeneous

- Commodity processors + GPU accelerators (or special AI processors)

Communication

- Special interconnects
- Moving data between parts very expensive compared to floating point operations

Floating point hardware

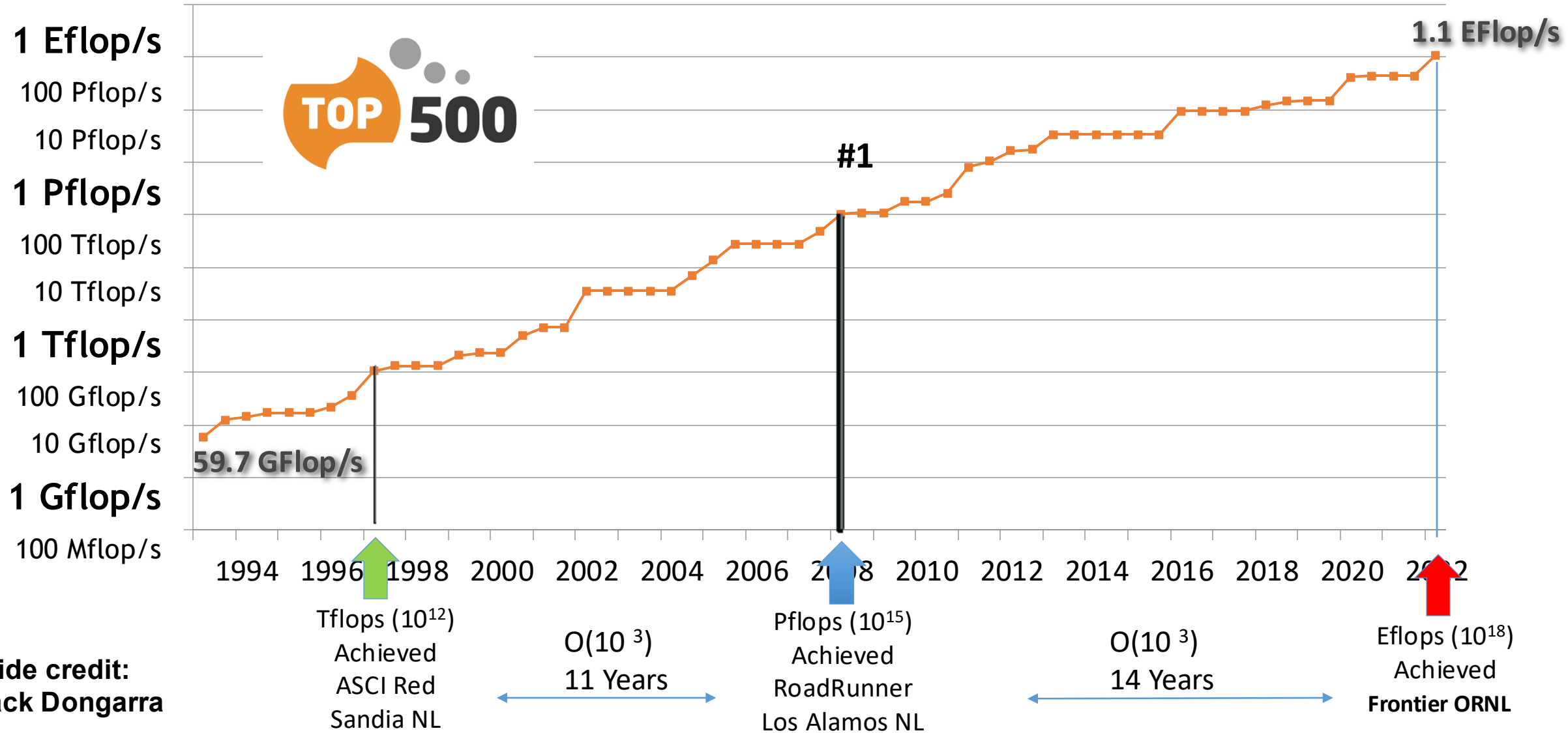
- 64, 32, 16, & 8 bit precision

LLNL El Capitan: 2.7 Eflop/s

11 million “cores”, 11,136 nodes, 30 MW
(node = 4 AMD APUs)

> 98% of performance from “GPUs”

Performance Development

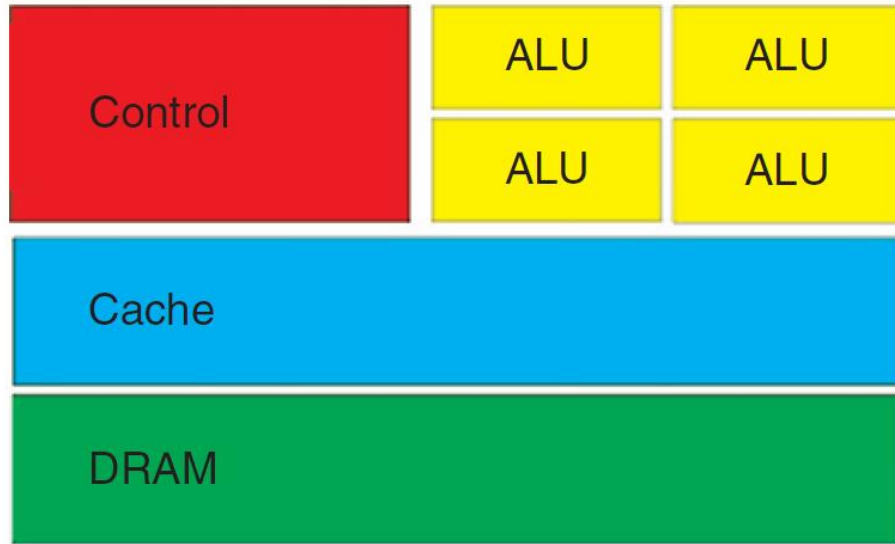


HPCG Benchmark Top 10, November 2022

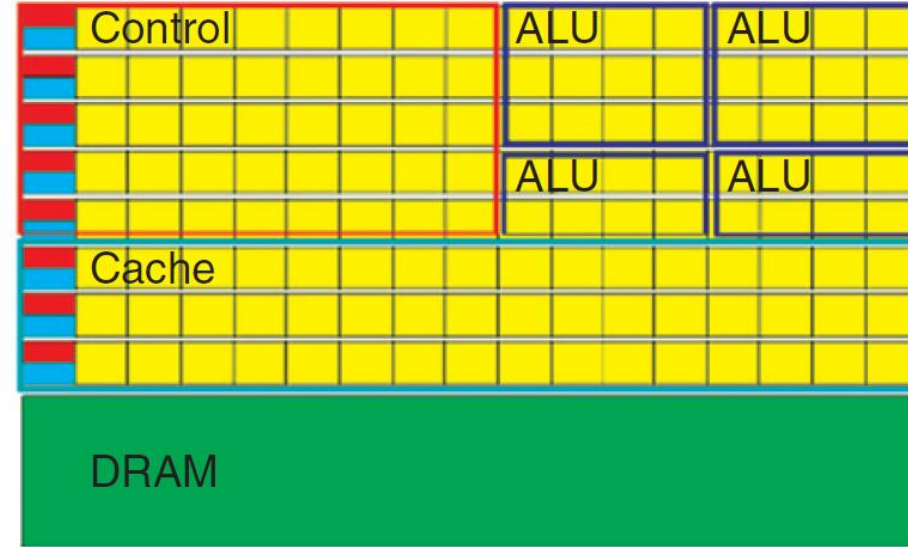
Rank	Site	Computer	Cores	HPL Rmax (Pflop/s)	TOP500 Rank	HPCG (Pflop/s)	Fraction of Peak
1	RIKEN Center for Computational Science Japan	Fugaku , Fujitsu A64FX 48C 2.2GHz, Tofu D, Fujitsu	7,630,848	442	2	16.0	3.0%
2	DOE/SC/ORNL USA	Frontier , HPE Cray Ex235a, AMD 3 rd EPYC 64C, 2 GHz, AMD Instinct MI250X, Slingshot 10	8,730,112	1,102	1	14.1	0.8%
3	EuroHPC/CSC Finland	LUMI , HPE Cray EX235a, AMD Zen-3 (Milan) 64C 2GHz, AMD MI250X, Slingshot-11	2,174,976	304	3	3.41	0.8%
4	DOE/SC/ORNL USA	Summit , AC922, IBM POWER9 22C 3.7GHz, Dual-rail Mellanox FDR, NVIDIA Volta V100, IBM	2,414,592	149	5	2.93	1.5%
5	EuroHPC/CINECA Italy	Leonardo, BullSequana XH2000, Xeon Platinum 8358 32C 2.6GHz, NVIDIA A100 SXM4 40 GB, Quad-rail NVIDIA HDR100 Infiniband	1,463,616	175	4	2.57	1.0%
6	DOE/SC/LBNL USA	Perlmutter , HPE Cray EX235n, AMD EPYC 7763 64C 2.45GHz, NVIDIA A100 SXM4 40 GB, Slingshot-10	761,856	70.9	8	1.91	2.0%
7	DOE/NNSA/LLNL USA	Sierra , S922LC, IBM POWER9 20C 3.1 GHz, Mellanox EDR, NVIDIA Volta V100, IBM	1,572,480	94.6	6	1.80	1.4%
8	NVIDIA USA	Selene , DGX SuperPOD, AMD EPYC 7742 64C 2.25 GHz, Mellanox HDR, NVIDIA Ampere A100	555,520	63.5	9	1.62	2.0%
9	Forschungszentrum Juelich (FZJ) Germany	JUWELS Booster Module , Bull Sequana XH2000 , AMD EPYC 7402 24C 2.8GHz, Mellanox HDR InfiniBand, NVIDIA Ampere A100, Atos	449,280	44.1	12	1.28	1.8%
10	Saudi Aramco Saudi Arabia	Dammam-7 , Cray CS-Storm, Xeon Gold 6248 20C 2.5GHz, InfiniBand HDR 100, NVIDIA Volta V100, HPE	672,520	22.4	20	0.88	1.6%

GPU vs CPU architecture

(a) CPU



(b) GPU



CPU

- Few processing cores with sophisticated hardware
- Multi-level caching
- Prefetching
- Branch prediction

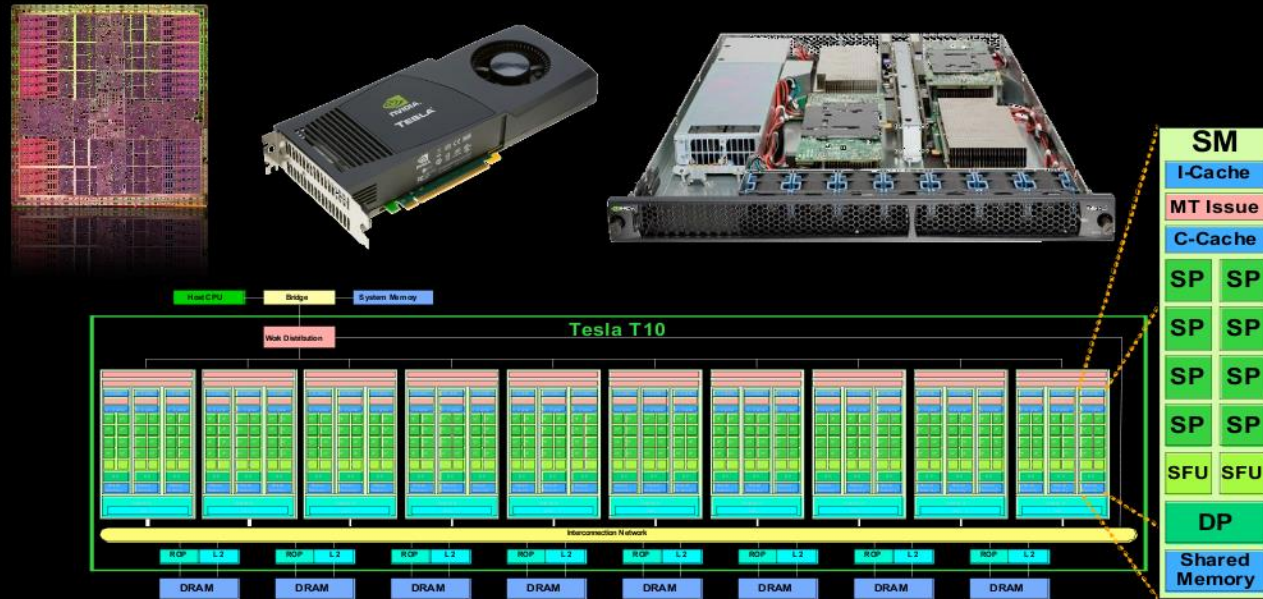
GPU

- Thousands of simplistic compute cores (packaged into a few multiprocessors)
- Operate in lock-step
- Vectorized loads/stores to memory
- Need to manage memory hierarchy

GPU architecture

CUDA Computing with Tesla T10

- 240 SP processors at 1.45 GHz: 1 TFLOPS peak
- 30 DP processors at 1.44GHz: 86 GFLOPS peak
- 128 threads per processor: 30,720 threads total



© NVIDIA Corporation 2008

Nvidia GPU architecture in 2009

- Tesla T10, a server with early C1060 datacenter GPU
- Basic architecture is still the same

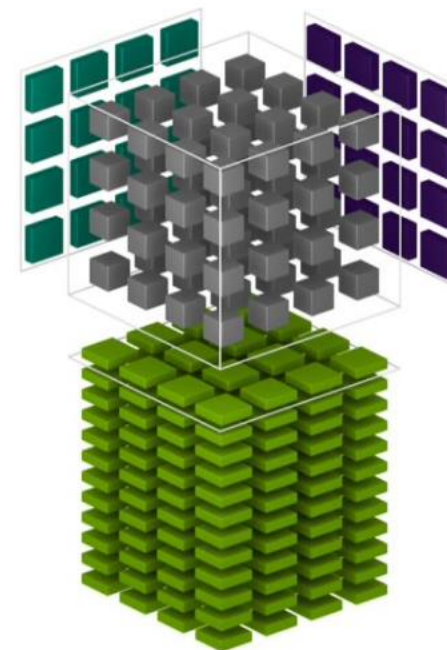
Multiprocessor

- FP32 compute cores
- FP64 compute core(s)
- Special function units
- Instruction cache
- Shared memory / data cache
- Handles many more threads than processing cores

Tensor cores

Accelerating MMA for FP64, TF32, Bfloat16, FP16, FP8

- Tensor Cores are specialized hardware for deep learning that help accelerate matrix multiply and accumulate (MMA) operations
- Nvidia Volta architecture introduced Tensor Cores with FP16 data types
- Nvidia Ampere GPUs introduce Tensor Core support for FP64, TF32, and Bfloat16 data types
- Nvidia Hopper GPUs introduced Tensor Core support for FP16 and FP8 data types
- Deep learning operations that benefit from tensor cores are
 - Fully connected / linear / dense layers
 - Convolutional layers
 - Recurrent layers
- Tensor Cores are also used for mixed precision matrix operations (CUBLAS library)



Tensor core 4x4 matrix multiply and accumulate

$$\begin{matrix} \text{FP16 or FP32} & \mathbf{D} = & \begin{pmatrix} \begin{matrix} A_{0,0} & A_{0,1} & A_{0,2} & A_{0,3} \\ A_{1,0} & A_{1,1} & A_{1,2} & A_{1,3} \\ A_{2,0} & A_{2,1} & A_{2,2} & A_{2,3} \\ A_{3,0} & A_{3,1} & A_{3,2} & A_{3,3} \end{matrix} & \begin{pmatrix} \begin{matrix} B_{0,0} & B_{0,1} & B_{0,2} & B_{0,3} \\ B_{1,0} & B_{1,1} & B_{1,2} & B_{1,3} \\ B_{2,0} & B_{2,1} & B_{2,2} & B_{2,3} \\ B_{3,0} & B_{3,1} & B_{3,2} & B_{3,3} \end{matrix} & + & \begin{pmatrix} \begin{matrix} C_{0,0} & C_{0,1} & C_{0,2} & C_{0,3} \\ C_{1,0} & C_{1,1} & C_{1,2} & C_{1,3} \\ C_{2,0} & C_{2,1} & C_{2,2} & C_{2,3} \\ C_{3,0} & C_{3,1} & C_{3,2} & C_{3,3} \end{matrix} \\ \text{FP16} & & \text{FP16 or FP32} \end{matrix}$$

SDSC Expanse – V100 SXM2 GPUs

Each GPU

- 32 GB HBM2 RAM (897 GB/s)
- 80 SMs (Streaming Multiprocessors)
- 64 FP32 cores / SM (5120 total)
- 32 FP64 cores / SM (2560 total)
- 8 Tensor cores / SM (640 total)
- 300 Watt TDP

Peak performance

- 7.8 FP64 TFLOPs
- 15.7 FP32 TFLOPs
- 31.3 FP16 TFLOPs
- 125 Tensor TFLOPs



GPU programming languages

CUDA

- Proprietary, works only for Nvidia GPUs
- De-facto standard for high-performance code

HIP

- Open-source C++ runtime API and kernel language developed by AMD
- Similar to CUDA, works with Nvidia (via CUDA) and AMD (via ROCm)

OpenCL

- Industry standard, works for Nvidia, AMD and Intel GPUs (and other devices)

SYCL

- Single-source, high-level, standard C++ programming model
- Can target a range of heterogeneous platforms (CPUs, GPUs, FPGAs)

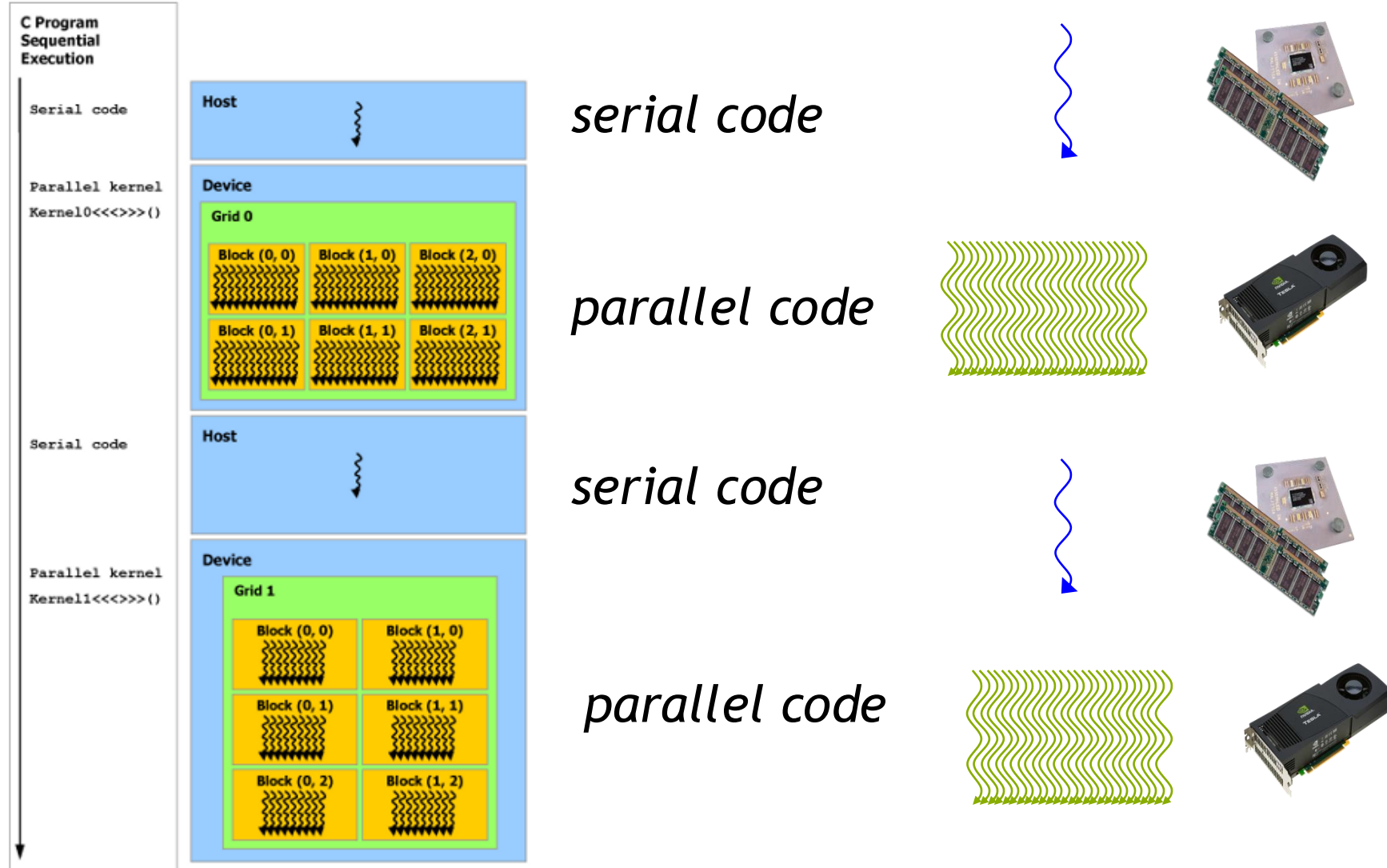
OpenACC

- Accelerator directives for Nvidia and AMD
- Works with C/C++ and Fortran

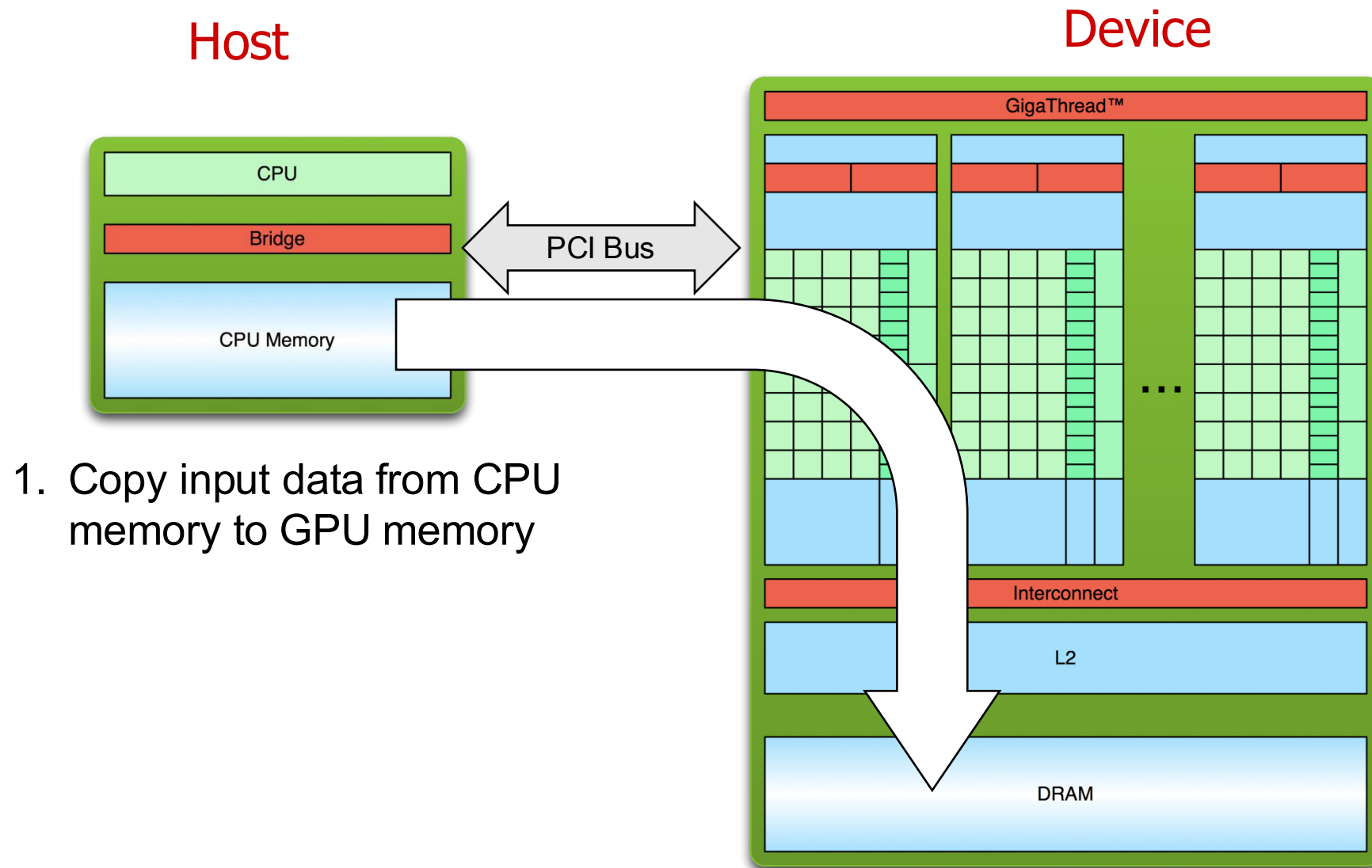
OpenMP

- Version 4.x includes accelerator and vectorization directives
- Becoming mature for GPUs

Heterogeneous Computing

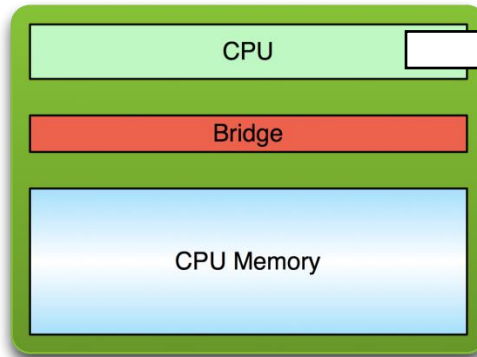


Processing Flow



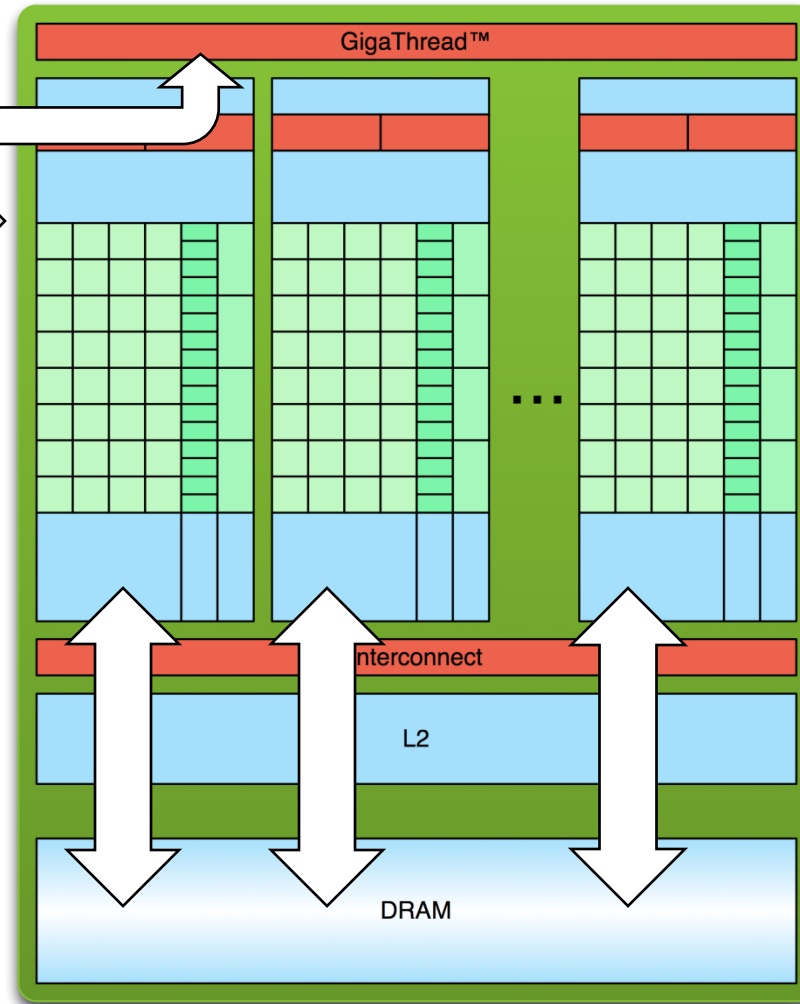
Processing Flow

Host



PCI Bus

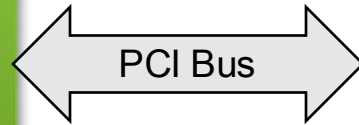
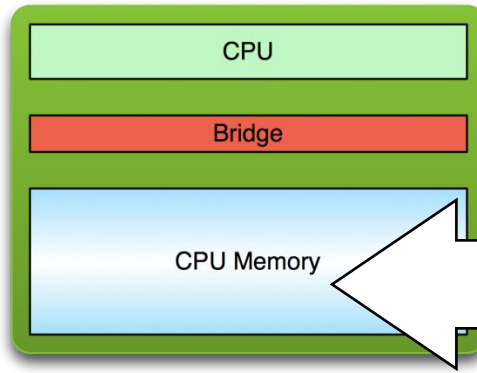
Device



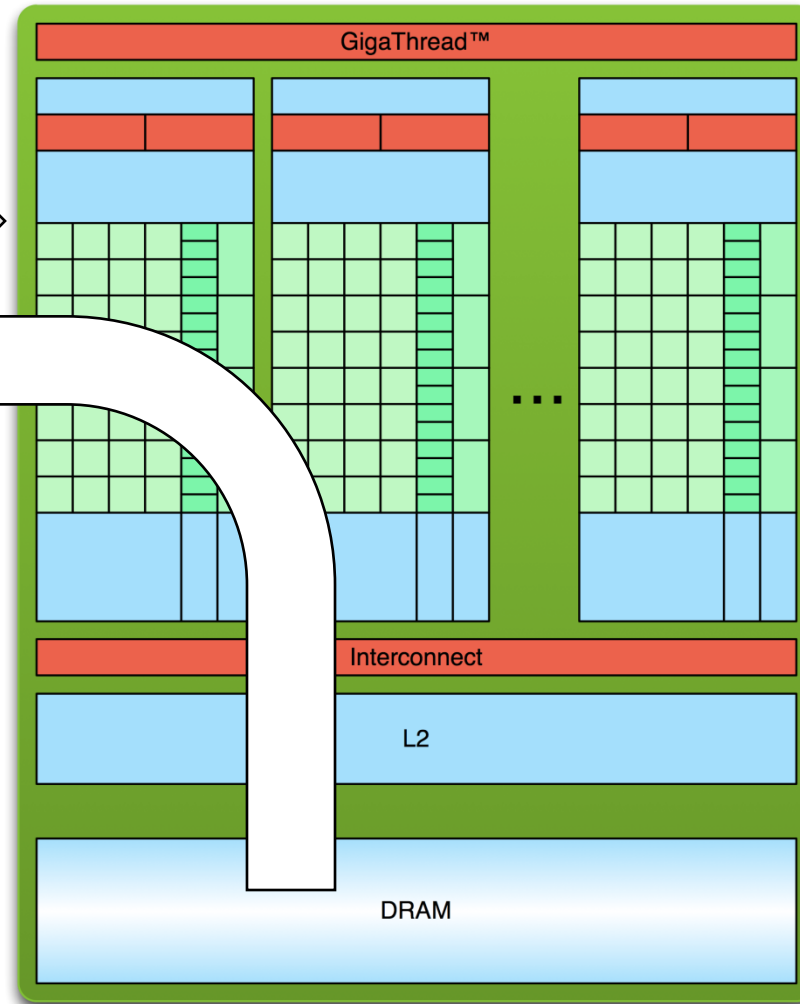
1. Copy input data from CPU memory to GPU memory
2. Load GPU program and execute, caching data on chip for performance

Processing Flow

Host

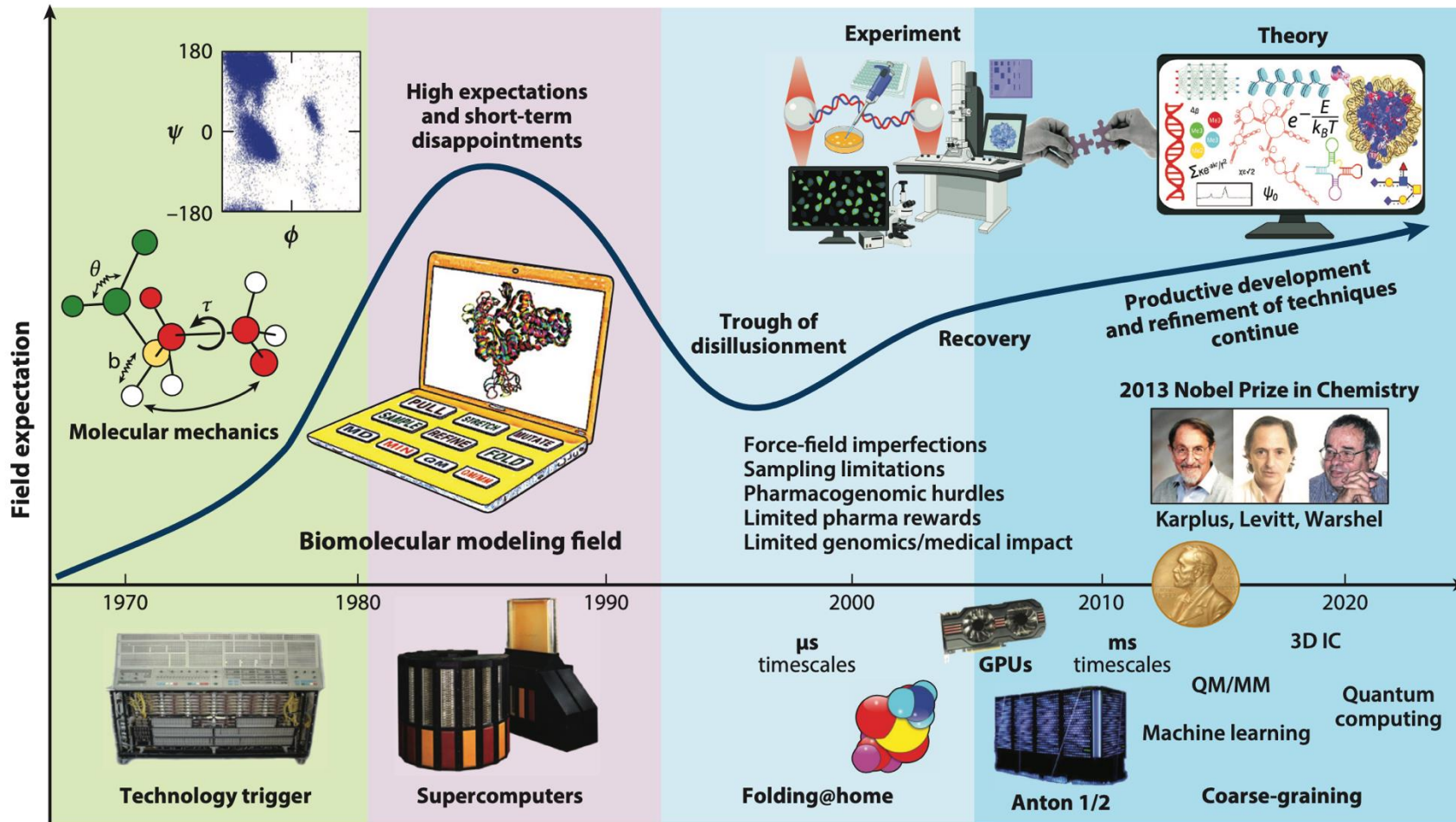


Device



1. Copy input data from CPU memory to GPU memory
2. Load GPU program and execute, caching data on chip for performance
3. Copy results from GPU memory to CPU memory

Biomolecular modeling over the years



Schlick et al. *Annu. Rev. Biophys.* **50** (2021) 267.

What is Amber?

A set of force field parameters

Currently recommended fixed charge force fields

- Proteins: ff19SB
- DNA and RNA: OL21 and OL3
- Lipids: lipid21
- Carbohydrates: GLYCAM_06j

Parameters for general organic molecules

- GAFF2 (General Amber Force Field version 2)

Experimental polarizable force fields

- E.g. ff02EP

Parameters for solvents and ions

What is Amber?

A set of force field parameters

Currently recommended fixed charge force fields

- Proteins: ff19SB
- DNA and RNA: OL21 and OL3
- Lipids: lipid21
- Carbohydrates: GLYCAM_06j

Parameters for general organic molecules

- GAFF2 (General Amber Force Field version 2)

Experimental polarizable force fields

- E.g. ff02EP

Parameters for solvents and ions

A molecular dynamics simulation package

- Around for over 35 years
- Approx. 50 principal contributors to current codes
- Independent of accompanying force fields (supports also CHARMM force field)
- Distributed in two parts

AmberTools

- Preparatory and analysis programs
- Molecular dynamics programs **sander**, **mdgx**
- Open source, mostly GPL

Amber

- High-performance MD program (MPI and CUDA)
- Academic license

Amber typical applications

Biomolecular simulations

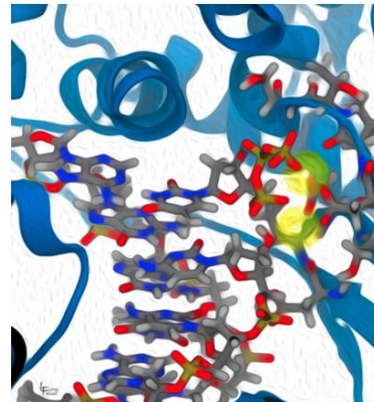
- Proteins, DNA, RNA, lipids, sugars, glycoproteins
- Building blocks of cells and tissue
- Understand how life works at the nanoscale
- Understand biomolecular function and disease
- Computational drug design

Molecular dynamics simulations

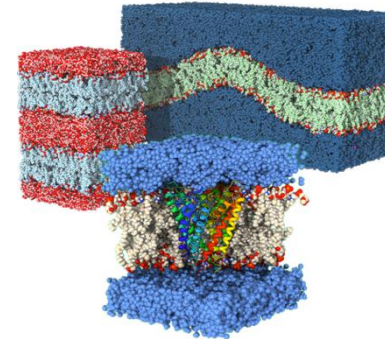
- Structure and function of biomolecular systems
- Protein folding
- Enzymatic reaction mechanisms
- Stability of protein / drug complexes
- Part of drug design pipeline (Improve drug leads via free energy simulations)
- And many more

<https://ambermd.org>

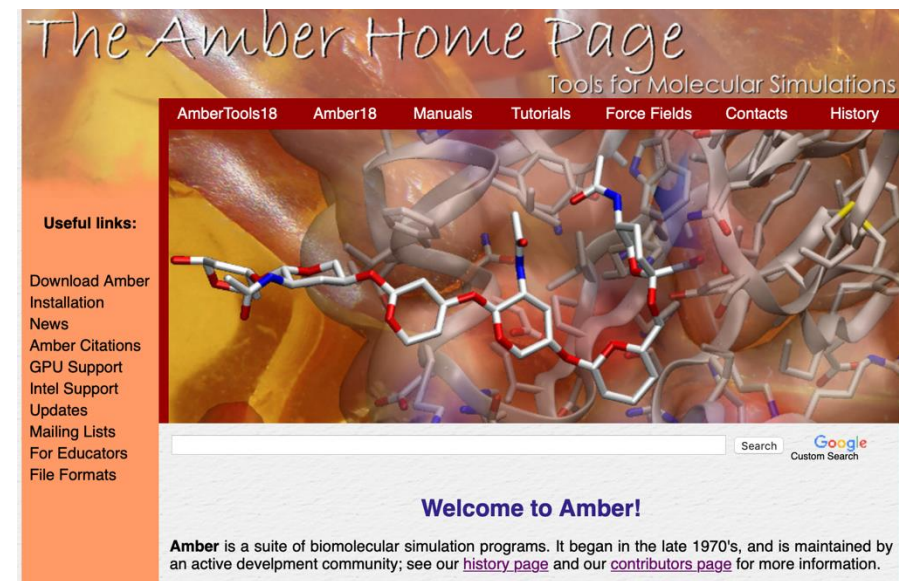
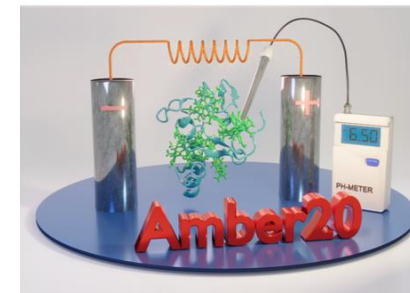
**Amber 2022
Reference Manual**
(Covers Amber22 and AmberTools22)



**Amber 2021
Reference Manual**
(Covers Amber20 and AmberTools21)



**Amber 2020
Reference Manual**
(Covers Amber20 and AmberTools20)



Amber is widely used

Amber user base

- Academic and industrial research
- User base steadily growing
- Installed on all major compute centers
- Popularity has been growing in particular since about 2010
 - Release of CUDA GPU code
 - Wide availability of GPUs enabled science for large user base

Amber licenses and AmberTools downloads since 2008.

Version	Year	Amber	AmberTools
10	2008	901	8,186
11	2010	969	10,210
12	2012	1,055	10,230
14	2014	995	12,203
16	2016	955	14,031
18	2018	1,032	16,034
20	2020	1,169	18,648

Routine microsecond molecular dynamics simulations with AMBER on GPUs. 2. Explicit solvent particle mesh Ewald

R Salomon-Ferrer, AW Götz, D Poole, S Le Grand, RC Walker
Journal of chemical theory and computation 9 (9), 3878-3888

Routine microsecond molecular dynamics simulations with AMBER on GPUs. 1. Generalized Born

AW Götz, MJ Williamson, D Xu, D Poole, S Le Grand, RC Walker
Journal of chemical theory and computation 8 (5), 1542-1555

3415 2013



2132 2012

PME GPU implementation
over 250 citations per year!

SPFP: speed without compromise—a mixed precision model for GPU accelerated molecular dynamics simulations

SL Grand, AW Götz, RC Walker
Computer Physics Communications

1156

2012

Google Scholar 6/4/2025

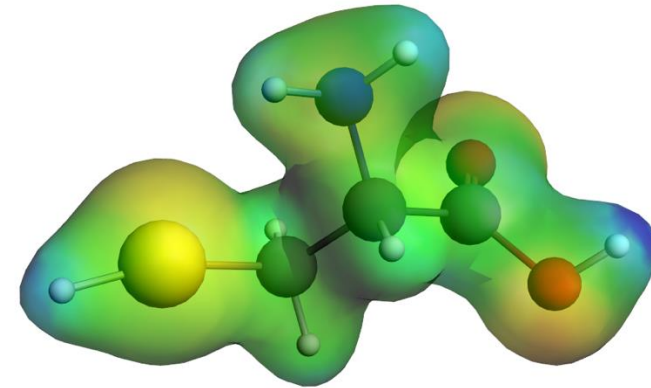
Modeling molecular interactions

Quantum Mechanics

$$\hat{H}\Psi = E\Psi$$

$$E[\rho] = T[\rho] + V_{ee}[\rho] + V_{xc}[\rho]$$

Solve Schrödinger equation
for many-electron system



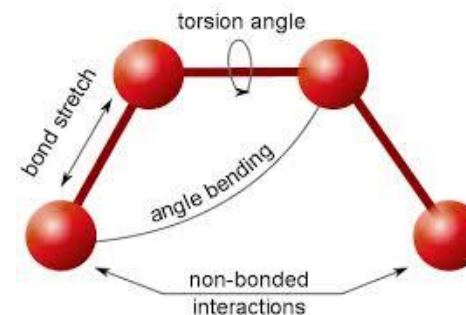
Modeling molecular interactions

Quantum Mechanics

$$\hat{H}\Psi = E\Psi$$

$$E[\rho] = T[\rho] + V_{ee}[\rho] + V_{xc}[\rho]$$

Simple “ball and stick model” for molecules



Molecular Mechanics Force Fields

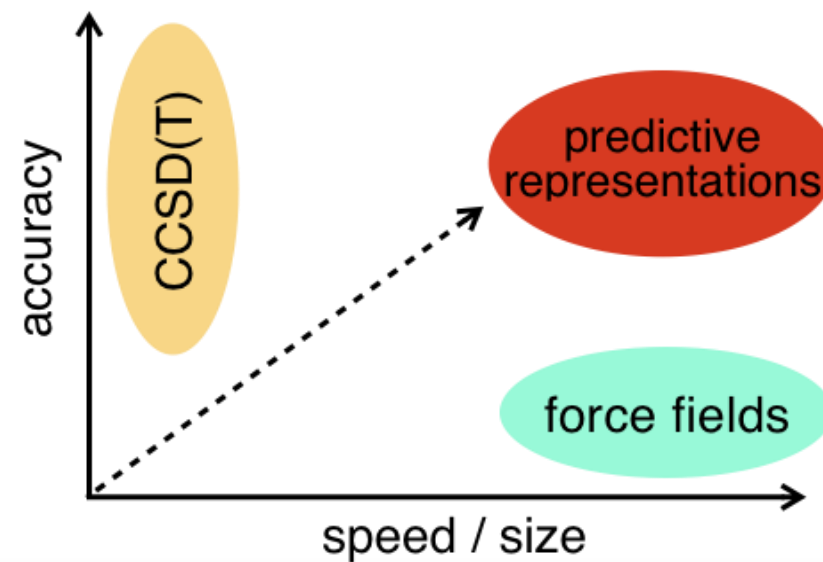
$$V(\vec{R}^N) = \sum_{bonds} \frac{k_b}{2} (r_b - r_{b,o})^2 + \sum_{angles} \frac{k_a}{2} (\theta_a - \theta_{a,o})^2 + \sum_{torsion} \left[\sum_n \frac{V_n}{2} (1 + \cos(n\omega - \gamma)) \right] \\ + \sum_{i < j} \left\{ 4\epsilon_{ij} \left[\left(\frac{\sigma_{ij}}{r_{ij}} \right)^{12} - \left(\frac{\sigma_{ij}}{r_{ij}} \right)^6 \right] + \frac{q_i q_j}{4\pi\epsilon_0 R_{ij}} \right\}$$

Modeling molecular interactions

Quantum Mechanics

$$\hat{H}\Psi = E\Psi$$

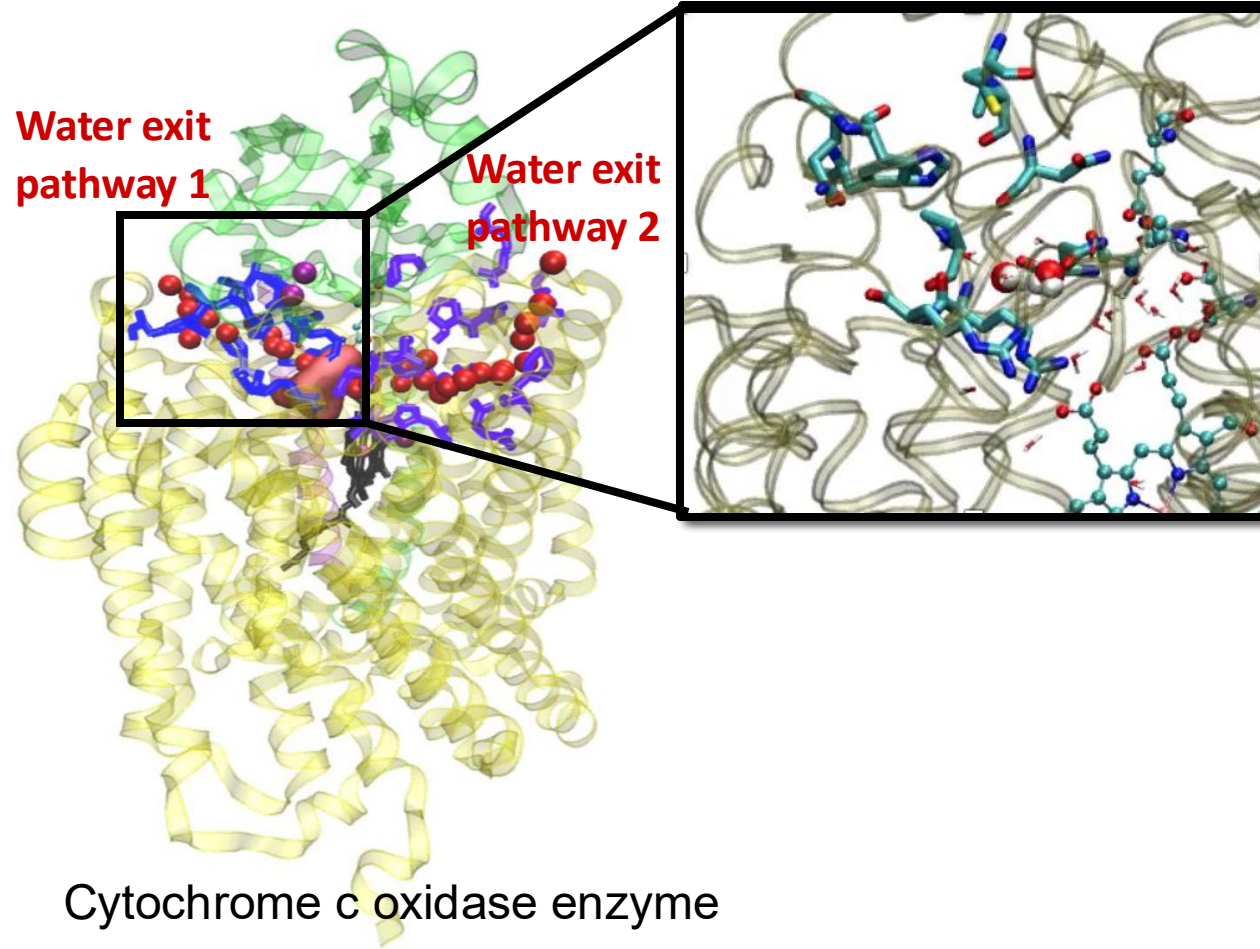
$$E[\rho] = T[\rho] + V_{ee}[\rho] + V_{xc}[\rho]$$



Molecular Mechanics Force Fields

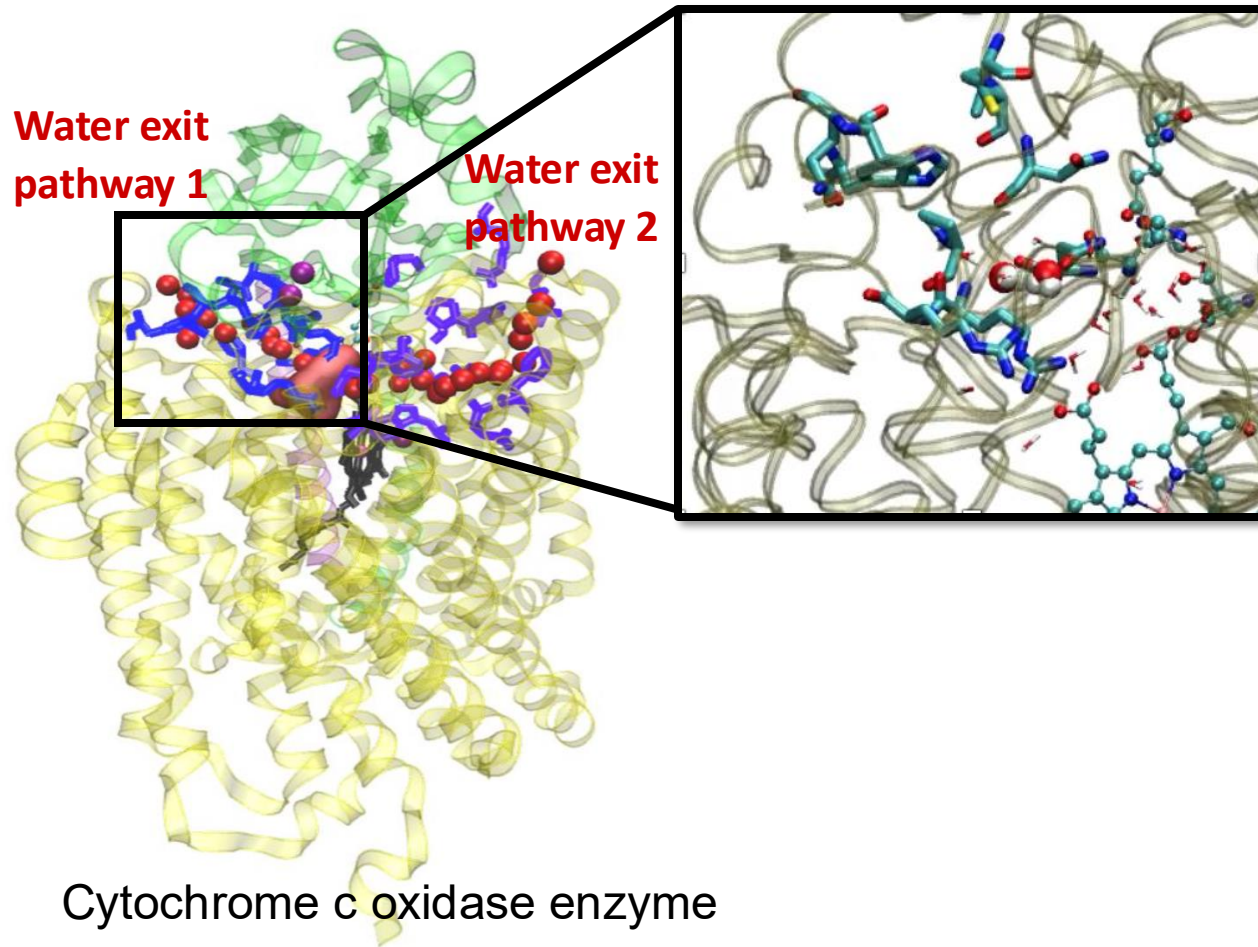
$$V(\vec{R}^N) = \sum_{bonds} \frac{k_b}{2} (r_b - r_{b,o})^2 + \sum_{angles} \frac{k_a}{2} (\theta_a - \theta_{a,o})^2 + \sum_{torsion} \left[\sum_n \frac{V_n}{2} (1 + \cos(n\omega - \gamma)) \right] \\ + \sum_{i < j} \left\{ 4\epsilon_{ij} \left[\left(\frac{\sigma_{ij}}{r_{ij}} \right)^{12} - \left(\frac{\sigma_{ij}}{r_{ij}} \right)^6 \right] + \frac{q_i q_j}{4\pi\epsilon_0 R_{ij}} \right\}$$

Molecular dynamics powered by GPUs

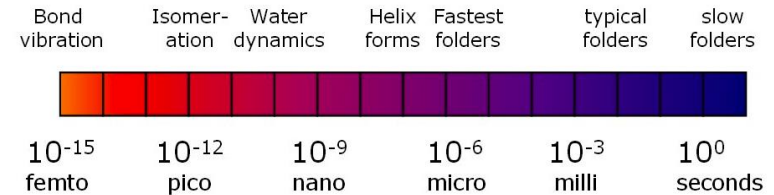


Yang, Skjevik, Han Du, Noodleman, Walker, Götz,
BBA Bioenergetics 2016 (1857) 1594.

Molecular dynamics powered by GPUs



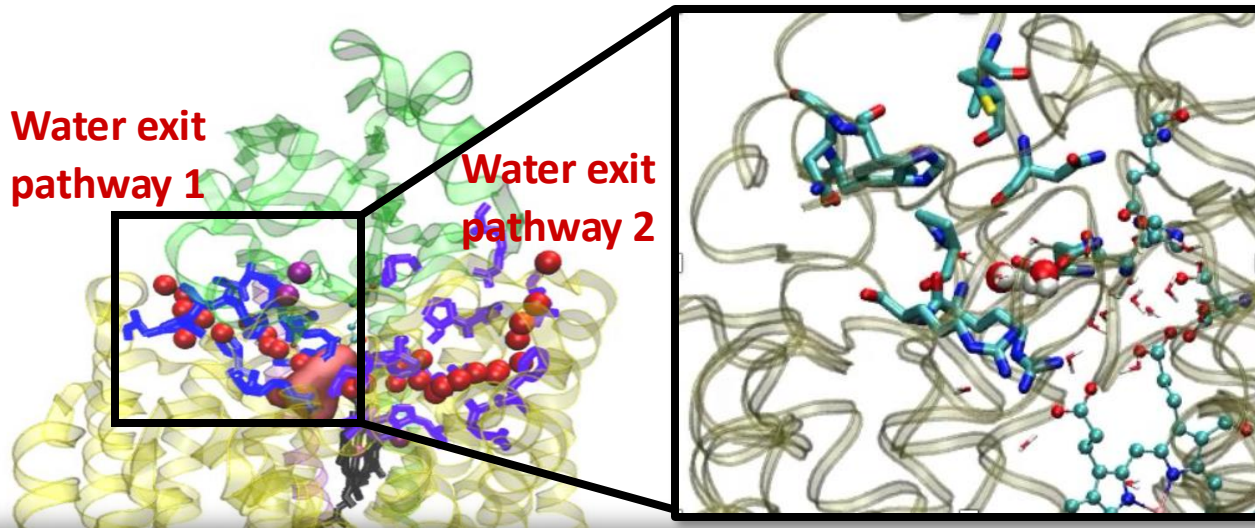
Relevant timescales



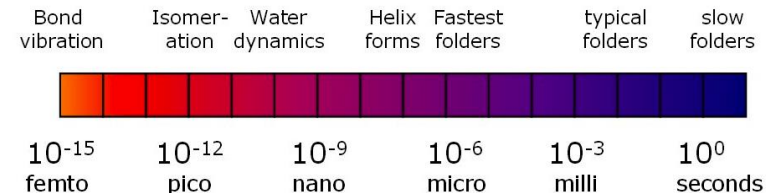
- Femtosecond timesteps
- Need to simulate micro to milliseconds
- 100s of millions of time steps required
- < 1 millisecond wall clock per time step

Yang, Skjevik, Han Du, Noodleman, Walker, Götz,
BBA Bioenergetics 2016 (1857) 1594.

Molecular dynamics powered by GPUs



Relevant timescales



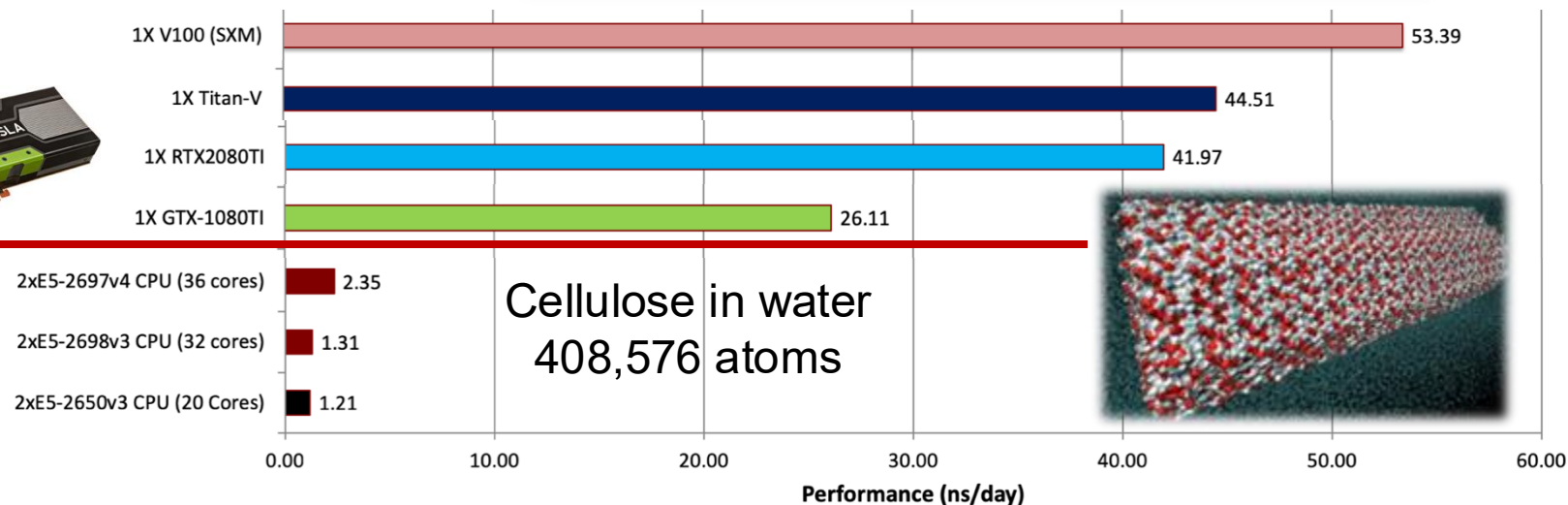
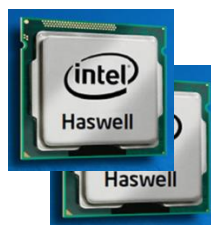
- Femtosecond timesteps
- Need to simulate micro to milliseconds
- 100s of millions of time steps required
- < 1 millisecond wall clock per time step

Amber 18 molecular dynamics software

Götz, Williamson, Xu, Poole, Le Grand, Walker, *J Chem Theory Comput* 2012 (8) 1542.

Le Grand, Götz, Walker, *Comput Phys Comm* 2013 (184) 374.

Salomon-Ferrer, Götz, Poole, Le Grand, Walker, *J Chem Theory Comput* 2012 (8) 1542.



QUICK – GPU-Enabled Quantum Chemistry

Distribution

- Open source
- Permissive license (Mozilla Public License 2.0)
- Download and contribute on Github
- Integrated and distributed with free and open-source AmberTools



Current Capabilities

- Hartree–Fock and DFT (GGA, hybrid-GGA)
- Gaussian basis sets (up to f functions)
- MPI parallel
- GPU enabled (all operations: OEIs, ERIs, XC quadrature, linear algebra)
- QM/MM (Amber for MM)

Get it for free on Github

<https://github.com/merzlab/quick>

Or via AmberTools

<https://ambermd.org>

Target Applications

- High-throughput QM (e.g. DFT energies/forces/atom charges for ML applications)
- QM/MM reaction profiles and free energy methods: drug design, PMF, binding free energies, etc

QUICK – Gaussian basis set Kohn-Sham DFT on GPUs

$$E[\rho] = T_s[\rho] + \int d\mathbf{r} \rho(\mathbf{r}) v_{\text{ext}}(\mathbf{r}) + \frac{1}{2} \int d\mathbf{r} d\mathbf{r}' \frac{\rho(\mathbf{r}) \rho(\mathbf{r}')}{|\mathbf{r} - \mathbf{r}'|} + E_{\text{xc}}[\rho]$$

Analytical electron repulsion integrals (ERIs)

- Coulomb interaction between electrons (3rd term)
- Exact exchange (not shown in equation above)

Numerical quadrature of XC potential and energy

- XC energy has complicated form
- Cannot be computed analytically
- Thousands of grid points per atom

$$E^{xc} = \int f(\rho_\alpha, \rho_\beta, \gamma_{\alpha\alpha}, \gamma_{\alpha\beta}, \gamma_{\beta\beta}) d\mathbf{r},$$

$$\int d\mathbf{r} f(\mathbf{r}) \approx \sum_i \omega_i f(\mathbf{r}_i)$$

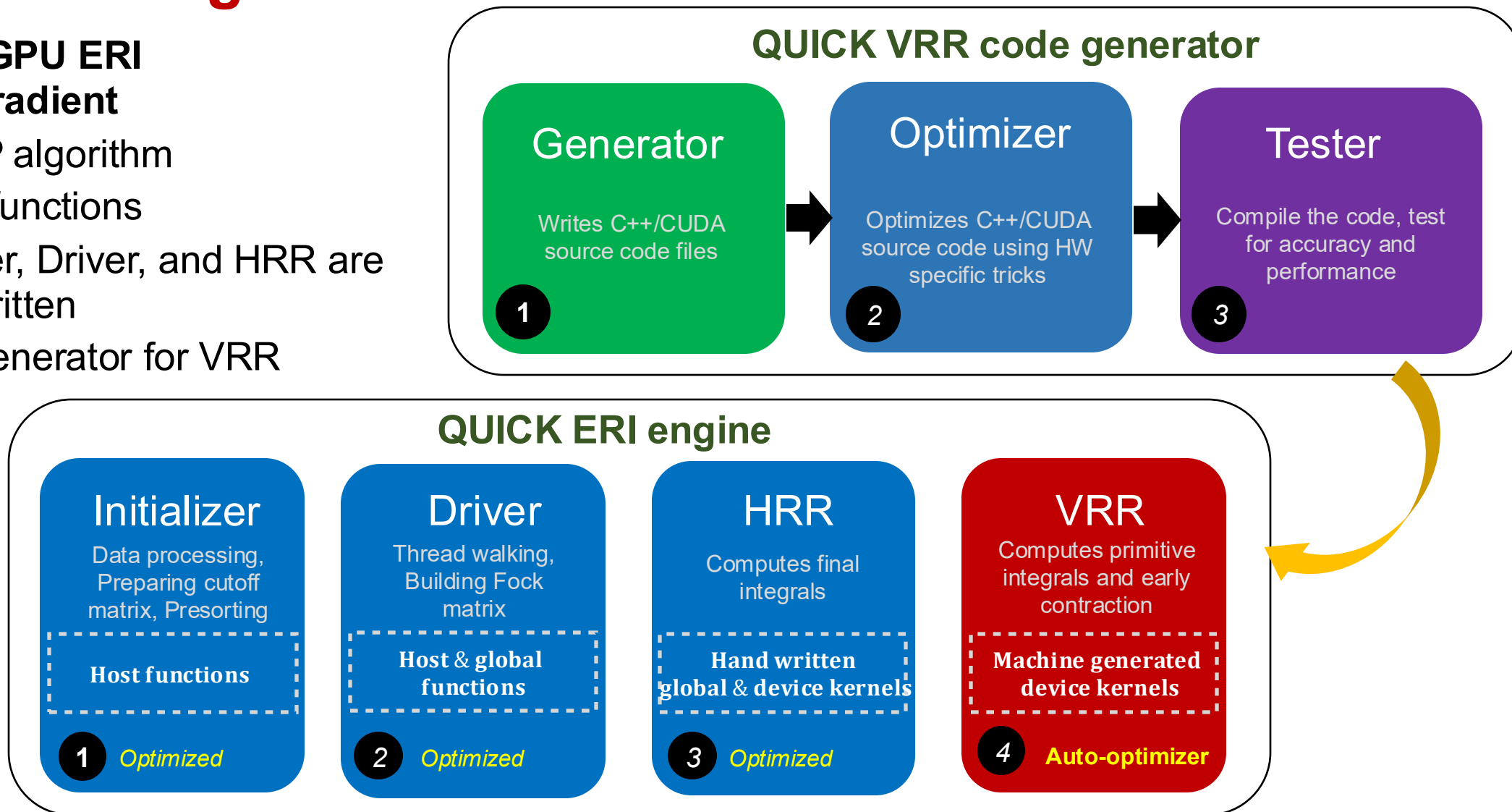
Relevant publications

- **OSHGP ERI algorithm for GPUs**
Miao, Merz, *JCTC* **11** (2015) 1449.
- **DFT octree-based quadrature on GPUs**
Manathunga, Miao, Mu, Götz, Merz, *JCTC* **16** (2020) 4315.
- **Multi-GPU parallelization**
Manathunga, Jin, Cruzeiro, Miao, Mu, Arumugam, Keipert, Aktulga, Merz, Götz, *JCTC* **17** (2021) 3955.
- **QM/MM implementation**
Cruzeiro, Manathunga, Merz, Götz, *JCIM* **61** (2021) 2109.
- **Nvidia and AMD GPU QM/MM + optimization**
Manathunga, Aktulga, Merz, Götz, *JCIM* **63** (2023) 711.
- **Geometry optimization**
Shajan, Manathunga, Götz, Merz, *JCTC* **19** (2023) 7533.

QUICK ERI Engine

CPU and GPU ERI and ERI gradient

- OSHGP algorithm
- Up to f functions
- Initializer, Driver, and HRR are hand-written
- Code generator for VRR

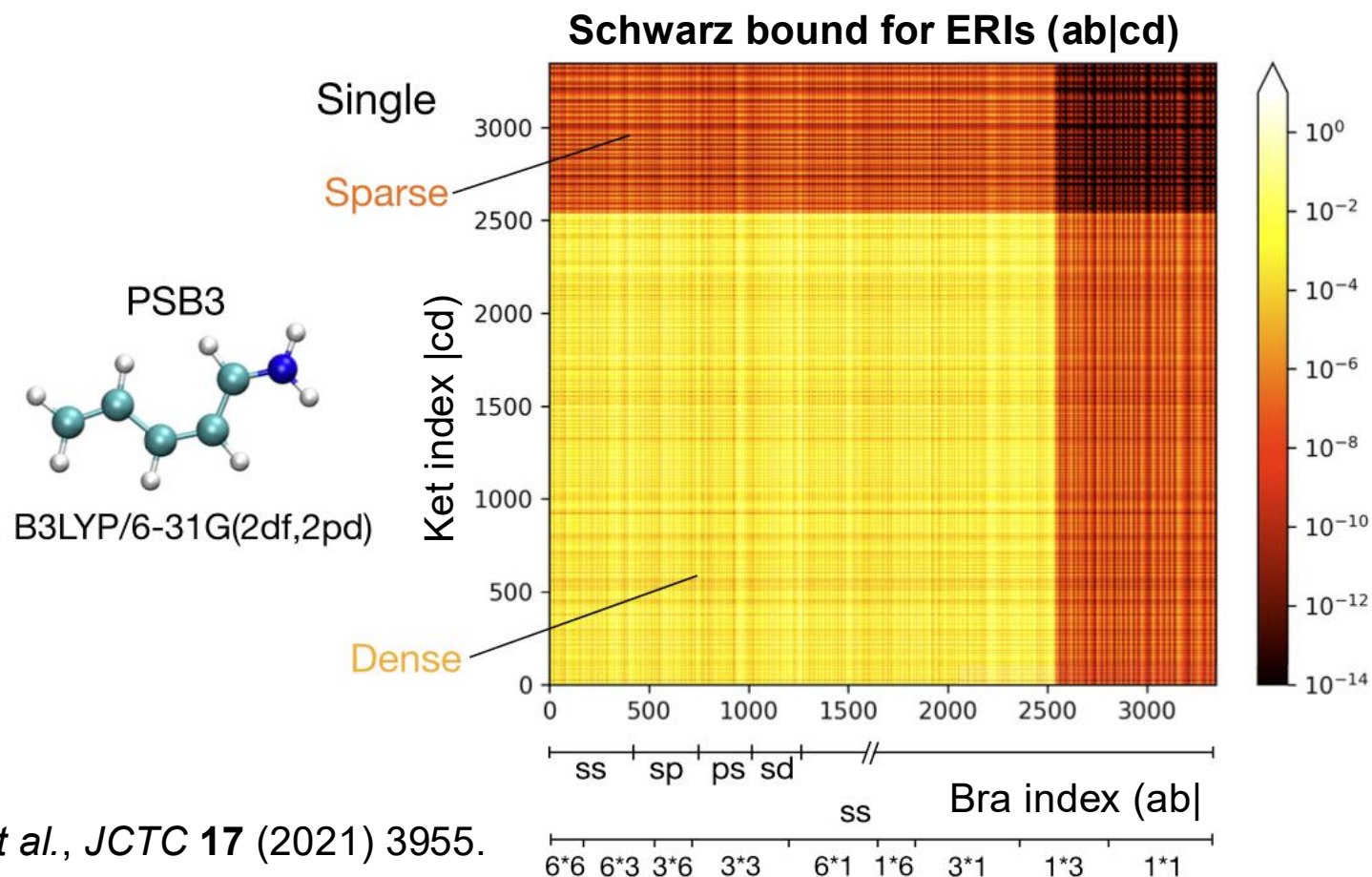


Parallelization of ERIs on GPUs

Sorting of ERIs is essential for performance on GPUs

- Size, angular momentum type, contraction
- Determines order in which ERIs are computed

- Neighboring threads thus do same work, avoiding thread divergence
- ERIs are computed via OSHGP recursion algorithm



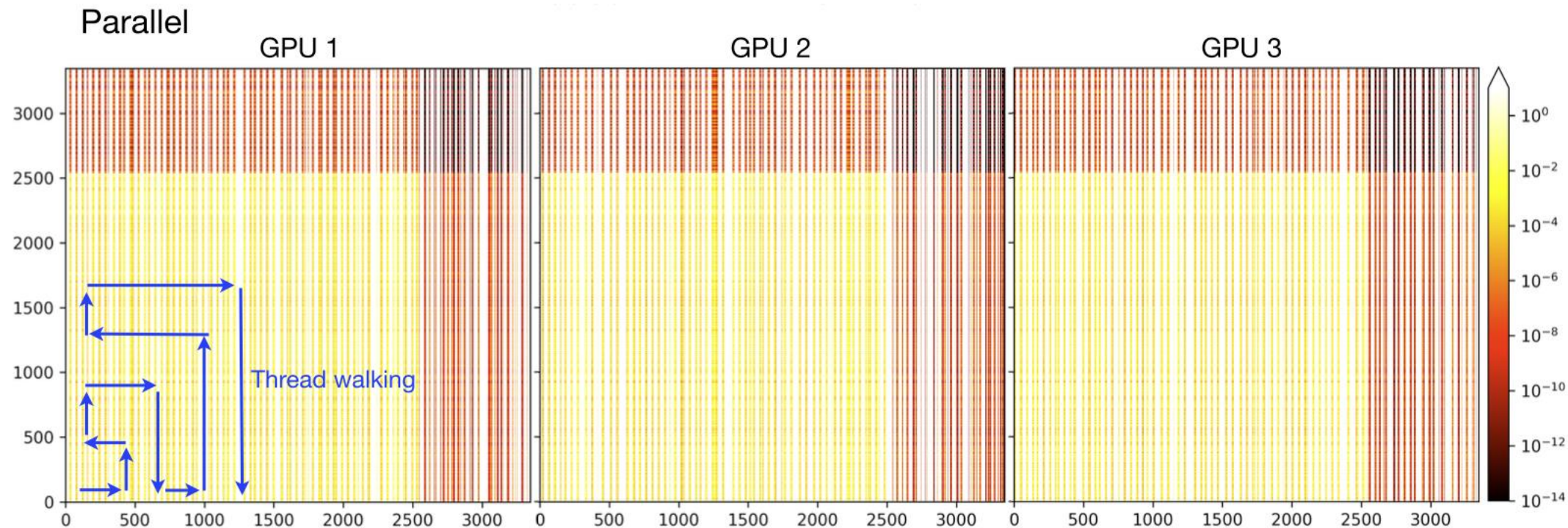
Zone	i+j	Range
Dense	0	0-401
	1	402-1035
	2	1036-1725
	3	1726-2204
	4	2205-2441
	5	2442-2523
	6	2523-2539
Sparse	0	2540-2666
	1	2667-2862
	2	2863-3091
	3	3092-3246
	4	3247-3325
	5	3326-3343
	6	3344-3347

Manathunga *et al.*, *JCTC* **17** (2021) 3955.

Parallelization of ERIs on GPUs

Sorted ERI workload is distributed across GPUs

- Bra types $|ab\rangle$ are distributed to different MPI ranks
- Each MPI rank controls one GPU
- Maintains load balance by distributing equal tasks

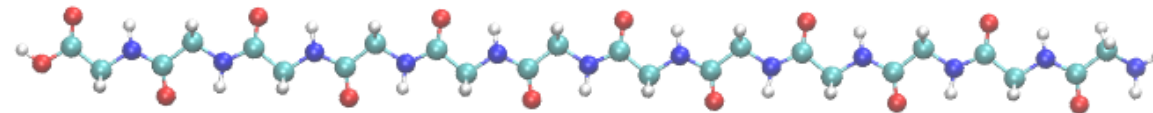


Manathunga *et al.*, *JCTC* **17** (2021) 3955.

Hartree-Fock accuracy and performance

Hartree-Fock benchmark

- (Glycine)₁₂
- t_{SCF} = time for first SCF iteration in seconds
- ΔE = Energy difference to QUICK GPU in 10^{-6} Hartree



Basis	N_{bas}	Quick GPU	Quick CPU		GAMESS		Psi4
		t_{SCF}	ΔE	t_{SCF}	ΔE	t_{SCF}	ΔE
6-31G	517		0.04		0.36		0.17
6-31G**	925		0.21		0.84		0.11
cc-pVDZ	925		0.06		0.67		0.06

Quick compares well

- Quick is as accurate as GAMESS and Psi4

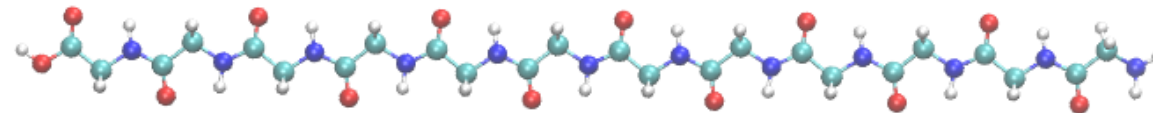
CPU: Intel Xeon E5-2680v4
GPU: Nvidia V100 (32GB)

Benchmarks by Madu Manathunga
QUICK 20.03 (new versions are much faster)

Hartree-Fock accuracy and performance

Hartree-Fock benchmark

- (Glycine)₁₂
- t_{SCF} = time for first SCF iteration in seconds
- ΔE = Energy difference to QUICK GPU in 10^{-6} Hartree



Basis	N_{bas}	Quick GPU	Quick CPU		GAMESS		Psi4
		t_{SCF}	ΔE	t_{SCF}	ΔE	t_{SCF}	ΔE
6-31G	517	1.3	0.04	16.8	0.36	16.6	0.17
6-31G**	925	3.8	0.21	72.2	0.84	84.8	0.11
cc-pVDZ	925	4.3	0.06	137.4	0.67	209.9	0.06

Quick compares well

- Quick is as accurate as GAMESS and Psi4
- Similar accuracy for gradients
- CPU implementation performs similar to GAMESS
- GPU implementation provides good speedup

CPU: Intel Xeon E5-2680v4
GPU: Nvidia V100 (32GB)

Benchmarks by Madu Manathunga
QUICK 20.03 (new versions are much faster)

Quick DFT XC Quadrature for CPU and GPU

Grid setup operations

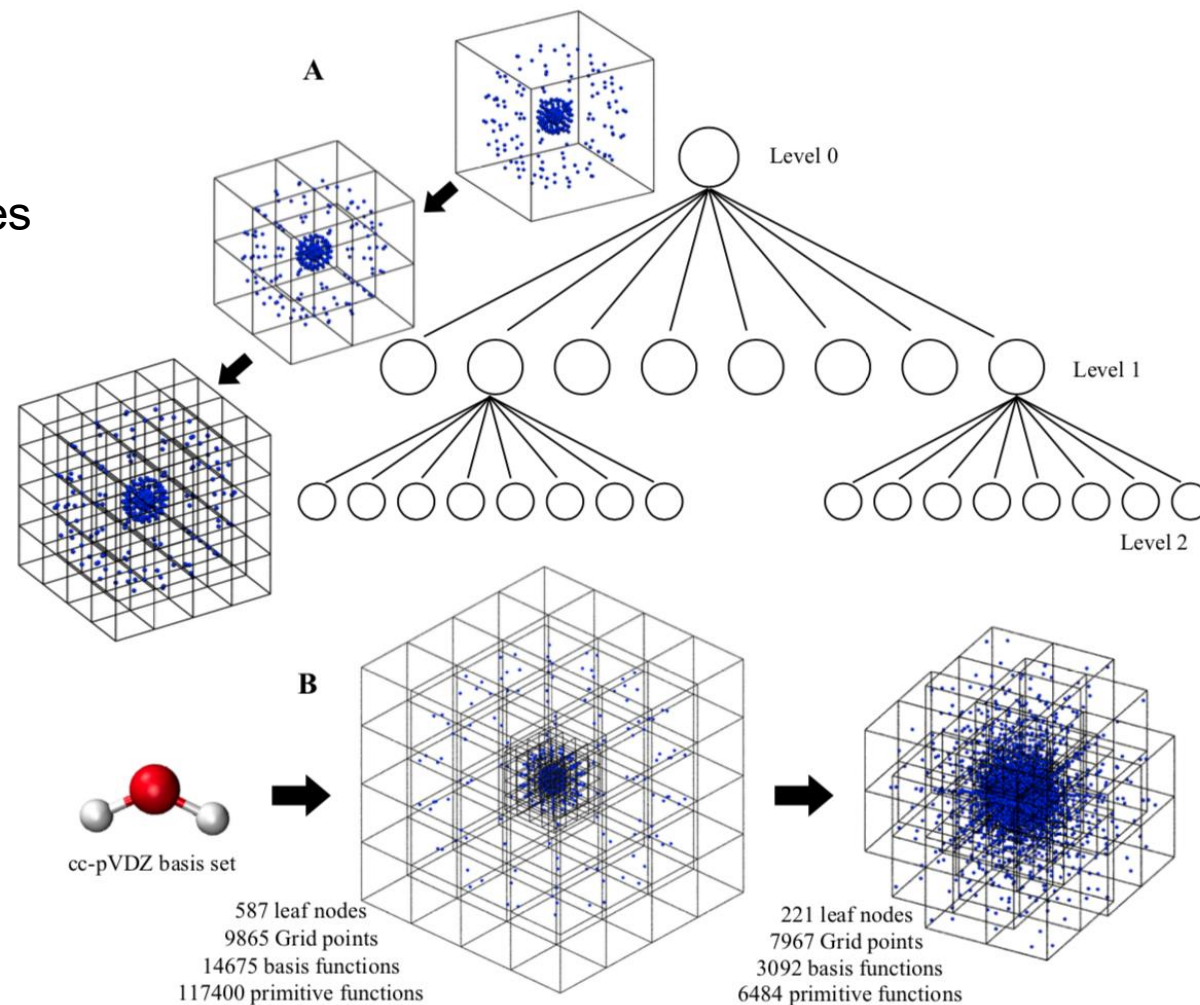
- Grid weight generation and pruning
- Octree based partitioning of 3D grid points
- Prescreening of function values on grid point batches leads to linear scaling for large molecules
- Preparation of basis & primitive functions list for each leaf node (bin)

XC quadrature

- Grid point batches are processed in parallel
- MPI for CPU
- CUDA or HIP/RocM for GPUs
- Assignment of bins (spatially local grid points) to thread blocks avoids thread divergence on GPUs

Manathunga, Miao, Mu, Götz, Merz, *JCTC* **16** (2020) 4315.

Octree based grid point partitioning



XC functionals: GPU port of libxc

Open-source libxc

- Gives access to practically all XC functionals
- QUICK currently supports GGA and hybrid-GGA functionals

Modified libxc for GPU (CUDA and HIP)

- Few additional source files mostly to handle upload of data structures required by libxc
- Minor modifications to existing source code using compiler directives

Builtin (not libxc) BLYP and B3LYP functionals

- Optimized for better performance on GPU than straight libxc port

Grimme dispersion via open-source DFTD library

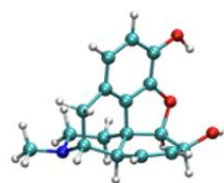
Manathunga, Miao, Mu, Götz, Merz, *JCTC* **16** (2020) 4315.

```
#ifdef DEVICE
__device__ void xc_gga_x_b88_enhance
(const void *p, xc_gga_work_x_t *r)
#else
void xc_gga_x_b88_enhance
(const xc_func_type *p, xc_gga_work_x_t *r)
#endif
{
    double t1, t2, t3, t4, t5, t7, t8, t9;
    double t10, t11, t12, t15, t16, t20, t24, t25;
    double t27, t28, t29, t32, t33, t37, t45, t46;
    double t47, t54, t57, t58, t72, t89;

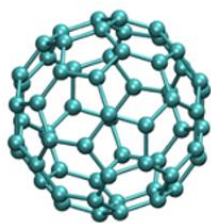
    gga_x_b88_params *params;
#ifdef DEVICE
    assert(p->params != NULL);
    params = (gga_x_b88_params * )(p->params);
```

Quick: multi-GPU scaling

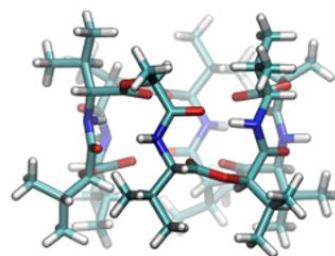
QUICK scaling across multiple GPUs (MPI + CUDA)



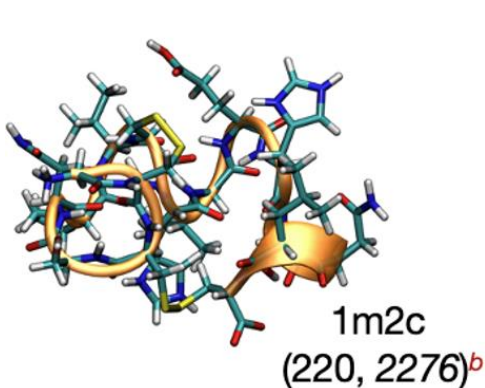
Morphine
(40, 410)^b



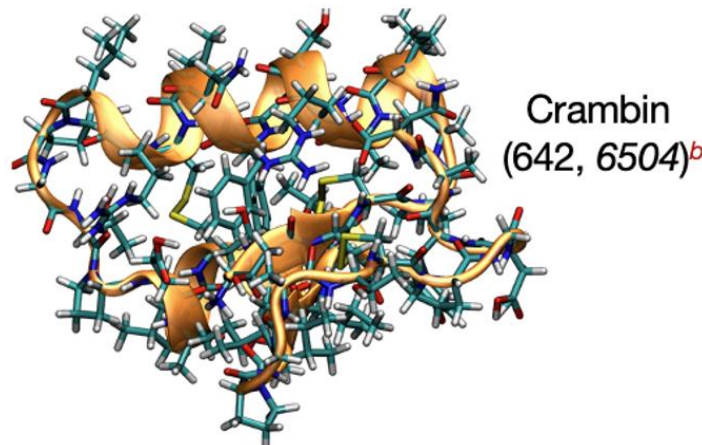
Buckyball
(60, 900)^b



Valinomycin
(168, 1620)^b



1m2c
(220, 2276)^b



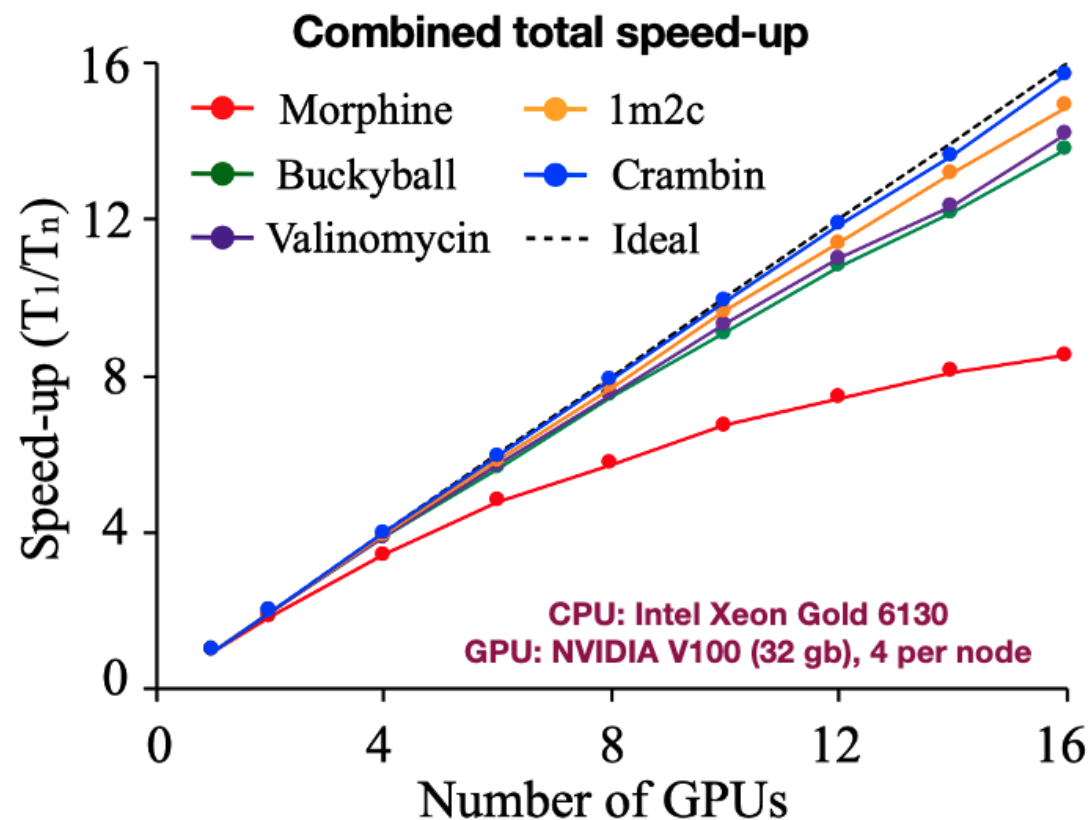
Crambin
(642, 6504)^b

b. The number of atoms and number of basis functions are given in brackets.

Benchmarks: QUICK 20.06

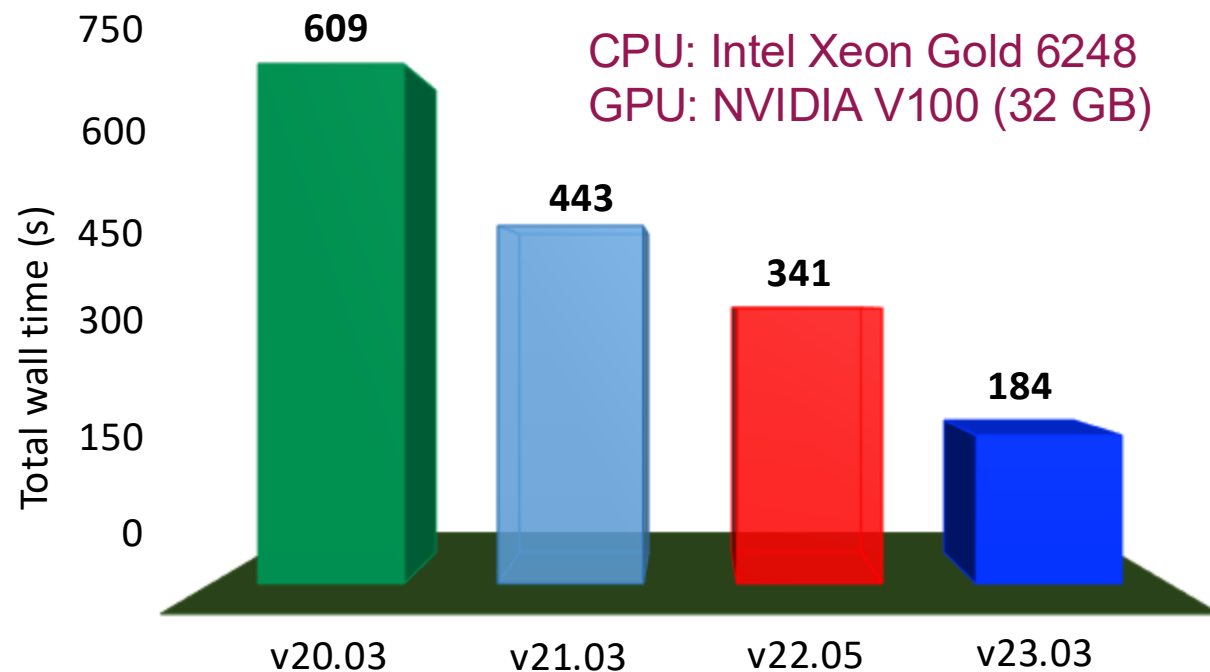
Manathunga *et al.*, *JCTC* **17** (2021) 3955.

B3LYP/6-31G** gradient calculations



QUICK performance evolution

Performance evolution of CUDA version



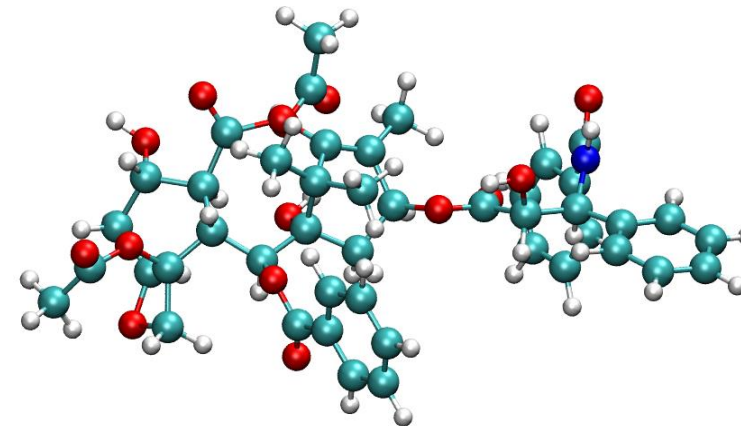
Note: Reported time includes SCF time and gradient time. Integral cutoff: 10^{-9} , density matrix RMS convergence: 10^{-8}

Improvements to ERI code generator

- Updated algorithms reduce memory bandwidth
- Current code is > 3x faster than benchmark data from pre-2022 publications

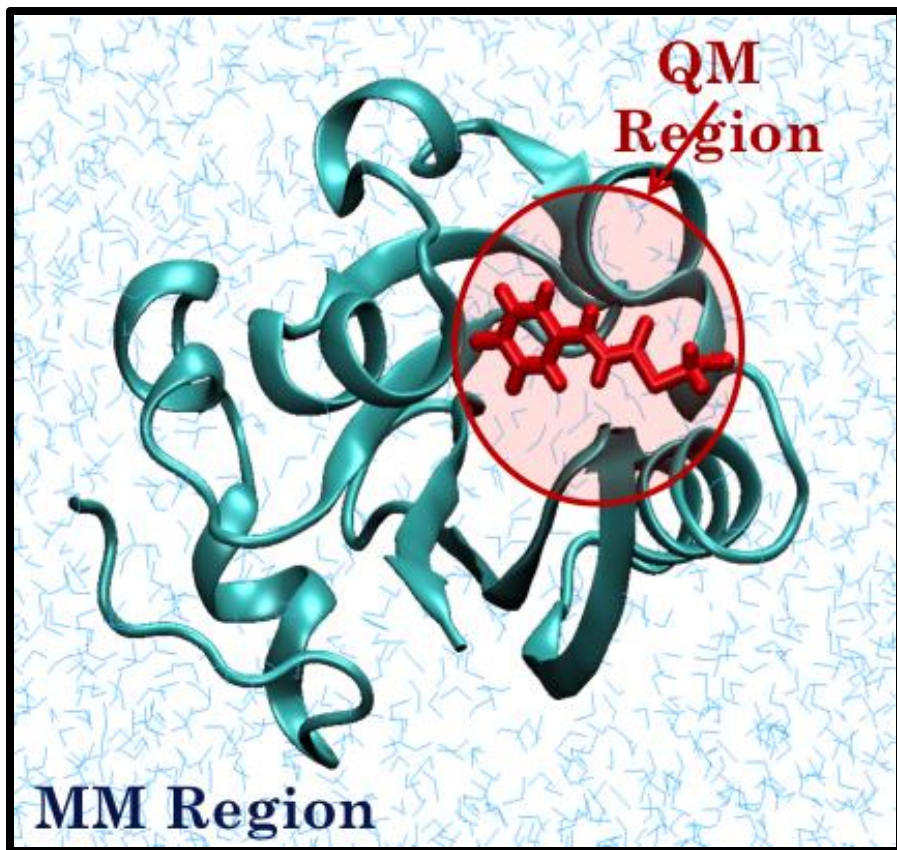
Work in progress

- ERI class-specific optimizations
- Mixed precision implementation (FP64/FP32) (expect ~2 to 3x speedup in upcoming releases)



Taxol B3LYP/6-31G**
(110 atoms, 1160 basis functions)

QUICK + AMBER: QM/MM



QUICK is distributed with AmberTools

- Standalone QUICK as QM program
- Integrated with sander via QM/MM API for QM/MM molecular dynamics simulations

Subdivide system:

- QM description of small “active” region
- MM description of environment
- Fixed partitioning
- Coupled potential

$$H = H_{QM} + H_{MM} + H_{QM/MM}$$

Warshel, Levitt, *J. Mol. Biol.* **103** (1976) 227.

Singh, Kollman, *J. Comput. Chem.* **7** (1986) 718.

Field, Bash, Karplus, *J. Comp. Chem.* **11** (1990) 700.

Senn, Thiel, *Angew. Chem. Int. Ed.* **48** (2009) 1198.

QUICK + AMBER: QM/MM

Data exchange

- QUICK QM/MM API
- Linked at source code level
- File-based for debugging or other codes

Amber sends

- QM settings (Method, basis, etc)
- QM atom coordinates
- MM atom coordinates and charges

QM code returns

- Energy
- Forces on QM atoms
- Forces on MM atoms due to electric field of QM charge density at MM atom positions

$$E = E_{\text{QM}} + E_{\text{MM}} + E_{\text{QM/MM}}$$

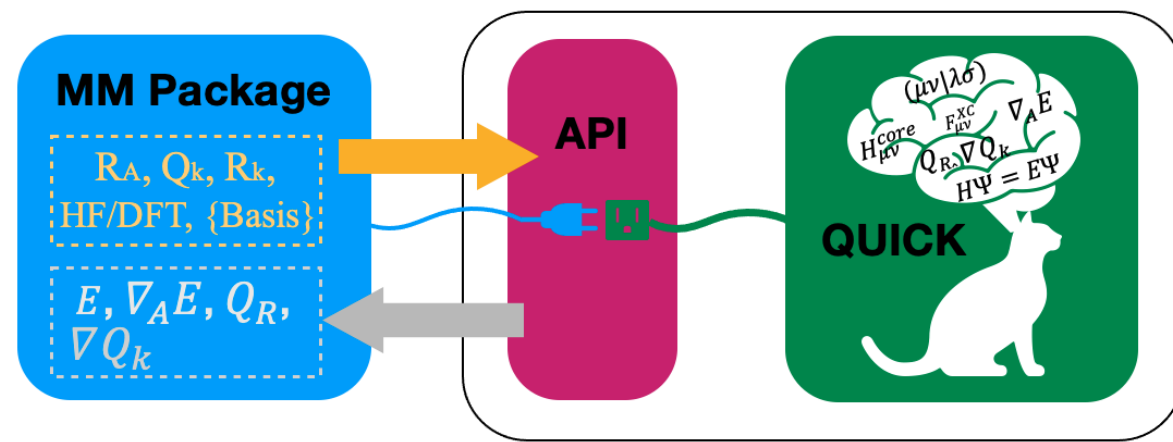
QM code MM code QM or MM code

Electrostatic embedding:

$$E_{\text{QM/MM}} = \sum_k^{N_{\text{MM}}} \int d\mathbf{r} \frac{\rho_{\text{QM}}(\mathbf{r}) Q_k}{|\mathbf{r} - \mathbf{R}_k|} + \sum_A^{N_{\text{QM}}} \sum_k^{N_{\text{MM}}} \epsilon_{Ak} \left[\left(\frac{\sigma_{Ak}}{R_{Ak}} \right)^{12} - \left(\frac{\sigma_{Ak}}{R_{Ak}} \right)^6 \right]$$

Mechanical embedding:

$$\rho_{\text{QM}}(\mathbf{r}) \longrightarrow \sum_A^{N_{\text{QM}}} Q_A \delta(\mathbf{r} - \mathbf{R}_A)$$



QUICK + AMBER: QM/MM

$$\mathbf{F} = -\nabla E_{\text{QM}} - \nabla E_{\text{MM}} - \nabla E_{\text{QM/MM}}$$

Forces on QM nuclei

$$\nabla_A E_{\text{QM/MM}} = \sum_k^{N_{\text{MM}}} Q_k \int d\mathbf{r} \frac{\nabla_A \rho_{\text{QM}}(\mathbf{r})}{|\mathbf{r} - \mathbf{R}_k|} + \sum_k^{N_{\text{MM}}} \nabla_A V_{Ak}^{\text{LJ}}$$

Forces on MM nuclei

$$\begin{aligned} \nabla_k E_{\text{QM/MM}} &= Q_k \int d\mathbf{r} \frac{\rho_{\text{QM}}(\mathbf{r}) \times (\mathbf{R}_k - \mathbf{r})}{|\mathbf{r} - \mathbf{R}_k|^3} + \sum_A^{N_{\text{QM}}} \nabla_k V_{Ak}^{\text{LJ}} \\ &= -Q_k \mathbf{E}_{\text{QM}}(\mathbf{R}_k) + \sum_A^{N_{\text{QM}}} \nabla_k V_{Ak}^{\text{LJ}} \end{aligned}$$

$$E = E_{\text{QM}} + E_{\text{MM}} + E_{\text{QM/MM}}$$

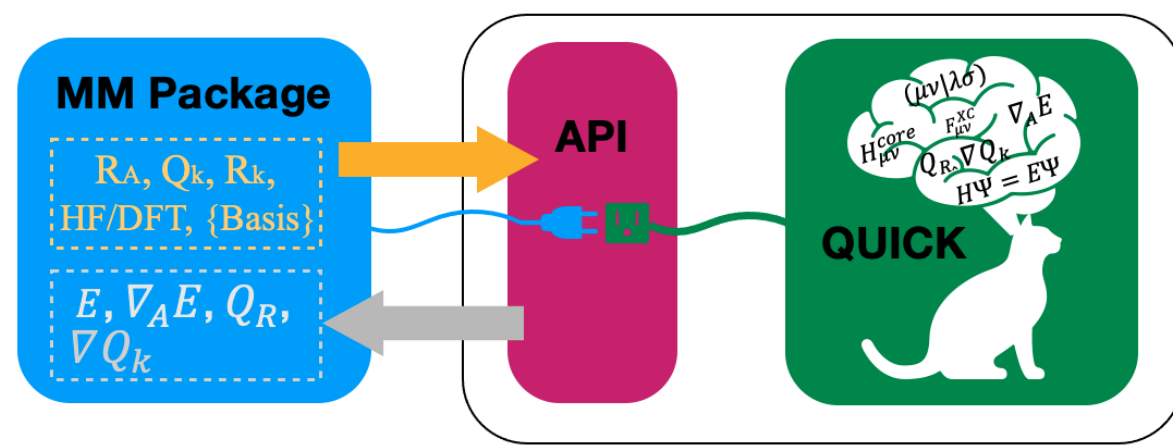
QM code MM code QM or MM code

Electrostatic embedding:

$$E_{\text{QM/MM}} = \sum_k^{N_{\text{MM}}} \int d\mathbf{r} \frac{\rho_{\text{QM}}(\mathbf{r}) Q_k}{|\mathbf{r} - \mathbf{R}_k|} + \sum_A^{N_{\text{QM}}} \sum_k^{N_{\text{MM}}} \epsilon_{Ak} \left[\left(\frac{\sigma_{Ak}}{R_{Ak}} \right)^{12} - \left(\frac{\sigma_{Ak}}{R_{Ak}} \right)^6 \right]$$

Mechanical embedding:

$$\rho_{\text{QM}}(\mathbf{r}) \longrightarrow \sum_A^{N_{\text{QM}}} Q_A \delta(\mathbf{r} - \mathbf{R}_A)$$



QM/MM covalent embedding

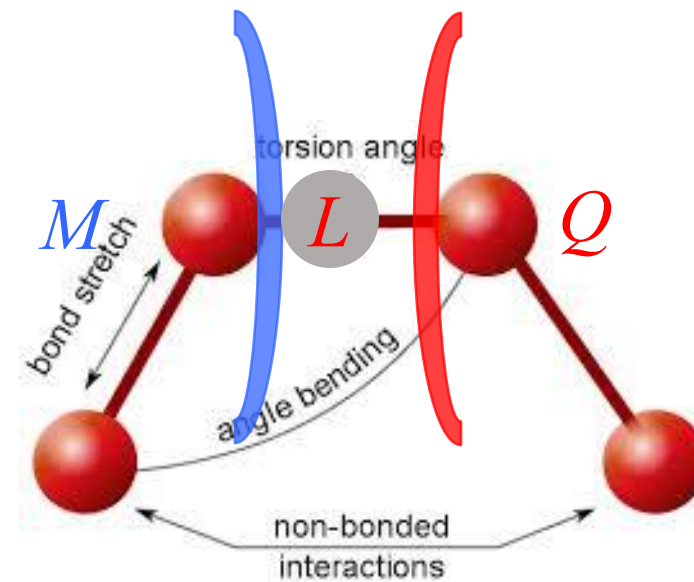
QM/MM covalent embedding

- QM/MM boundary crosses covalent bond
- Plethora of ways to deal with...

Amber implements automatic link atom scheme

- Hydrogen link atoms at fixed distance along broken bond
- Bonded terms (bonds, angles, dihedrals) removed if all atoms in QM region
- FF vdW terms not modified
- Charge on MM link atom removed for electrostatic embedding, excess charge distributed onto all remaining MM atoms
- Simple but robust approach if QM/MM boundary chosen wisely (e.g. do not cut polar bonds or double bonds)

Link atom scheme



$$\mathbf{r}_L = \mathbf{r}_Q + b_{LQ} \frac{\mathbf{r}_M - \mathbf{r}_Q}{|\mathbf{r}_M - \mathbf{r}_Q|}$$

Position with respect to QM boundary atom Q and MM boundary atom M, does not introduce new degrees of freedom.

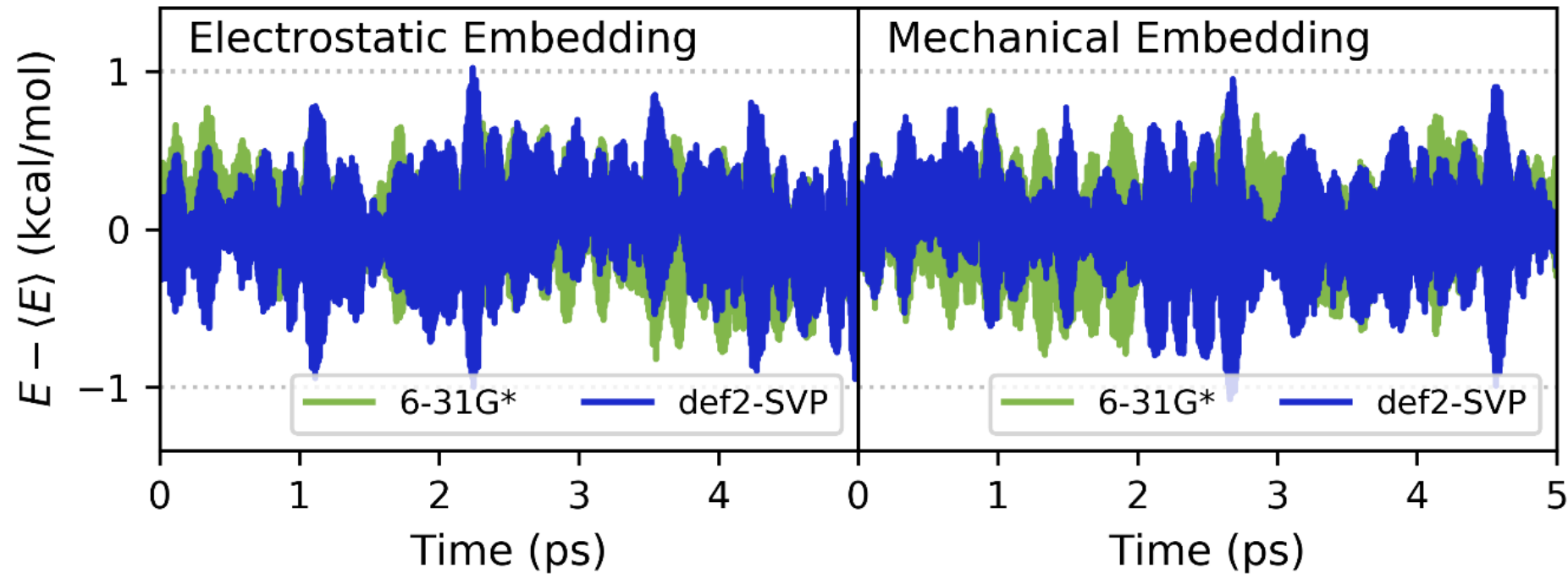
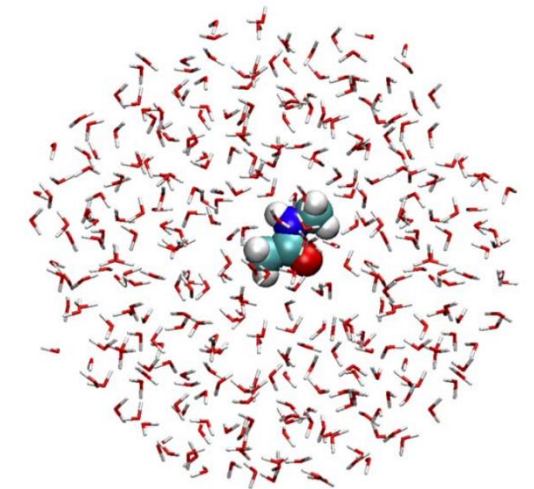
QM/MM via AMBER/QUICK

Implementation is numerically stable

- Good energy conservation with default SCF settings

NMA in water

- NMA treated with B3LYP
- Water SPC/Fw



Cruzeiro, Manathunga, Merz, Götz, *JCIM* **61** (2021) 2109.

QM/MM Ewald / PME long-range electrostatics semiempirical / tight binding models

Model QM charge density via Mulliken charges

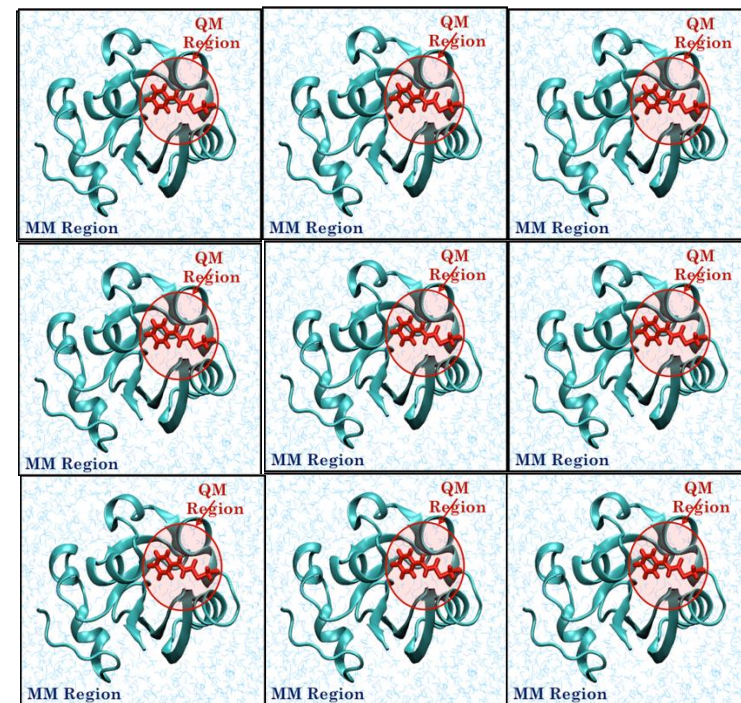
- Semiempirical NDDO methods
- GFN1-xTB and GFN2-xTB
- DFTB

$$E_{\text{QM/MM}} = E_{\text{QM/MM,LJ}} + \sum'_{b \in \text{MM}} \iint \frac{q_{\text{QM}}(\mathbf{r}') q_b(\mathbf{r} - \mathbf{r}_b)}{|\mathbf{r} - \mathbf{r}'|} d^3 r d^3 r'$$

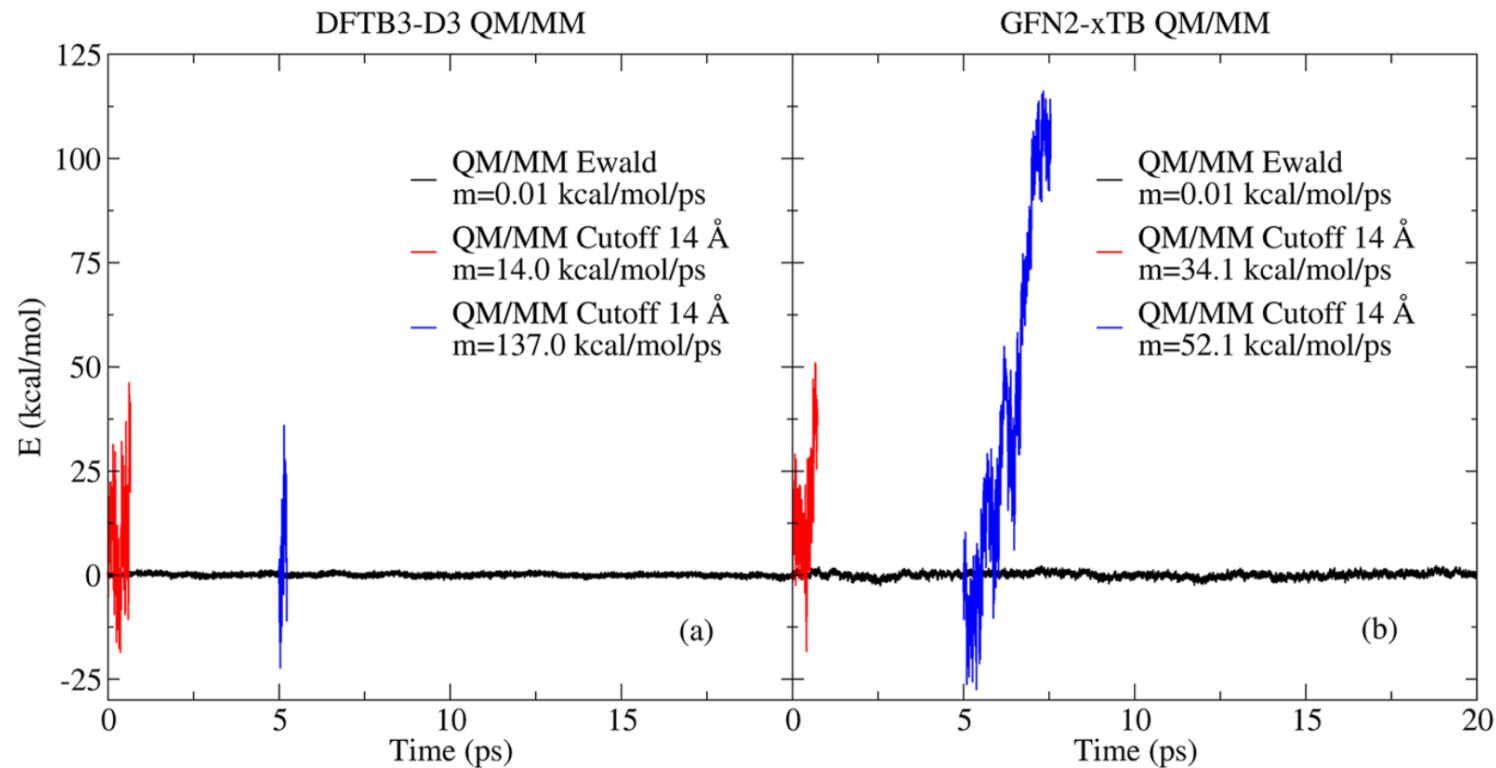
Long range electrostatics $\longrightarrow + \sum_{a \in \text{QM}} q_a \left(\Delta\phi_{\text{MM}}(\mathbf{r}_a) + \frac{1}{2} \Delta\phi_{\text{QM}}(\mathbf{r}_a) \right)$

$$\Delta\phi_{\text{MM}}(\mathbf{r}_a) = \text{Re} \sum_{\mathbf{k} \neq 0} \frac{4\pi}{k^2 V} e^{i\mathbf{k}^T \cdot \mathbf{r}_a - \frac{k^2}{4\beta^2}} \sum_{b \in \text{MM}} q_b e^{-i\mathbf{k}^T \cdot \mathbf{r}_b} - \sum'_{b \in \text{MM}} \frac{q_b}{r_{ab}} \frac{\text{erf}(\beta r_{ab})}{r_{ab}} - \frac{\pi Q_{\text{MM}}}{\beta^2 V},$$

$$\Delta\phi_{\text{QM}}(\mathbf{r}_a) = \text{Re} \sum_{\mathbf{k} \neq 0} \frac{4\pi}{k^2 V} e^{i\mathbf{k}^T \cdot \mathbf{r}_a - \frac{k^2}{4\beta^2}} \sum_{b \in \text{QM}} q_b e^{-i\mathbf{k}^T \cdot \mathbf{r}_b} - \sum_{b \in \text{QM}} \frac{q_b}{r_{ab}} \frac{\text{erf}(\beta r_{ab})}{r_{ab}} - \frac{\pi Q_{\text{QM}}}{\beta^2 V}.$$



QM/MM Ewald / PME long-range electrostatics semiempirical / tight binding models



Tight binding models

- Cutoff leads to energy drift
- PME with Mulliken charges is numerically stable

FIG. 2. QM/MM energy conservation of B-DNA in the microcanonical ensemble. (a) DFTB3-D3 performed with Sander/DFTB+. (b) GFN2-xTB performed with Sander/xTB. All methods compute the MM/MM electrostatics with particle mesh Ewald. The QM/MM Ewald method similarly models the QM/QM and QM/MM electrostatics with Ewald. The QM/MM cutoff 14 Å method truncates the QM/MM interactions beyond 14 Å, and long-range QM/QM interactions are ignored. The QM/MM cutoff 14 Å simulations were repeated at different starting configurations taken from the QM/MM Ewald trajectory. The “m” values are the slope of the energy obtained from linear regression.

Tao, Giese, Ekesan, Zheng, Aradi, Hourahine, Aktulga, Götz, Merz, York, *J. Chem. Phys.* **160** (2024) 224104.

QM/MM Ewald / PME long-range electrostatics – HF/DFT

Ambient Potential Composite Ewald Method

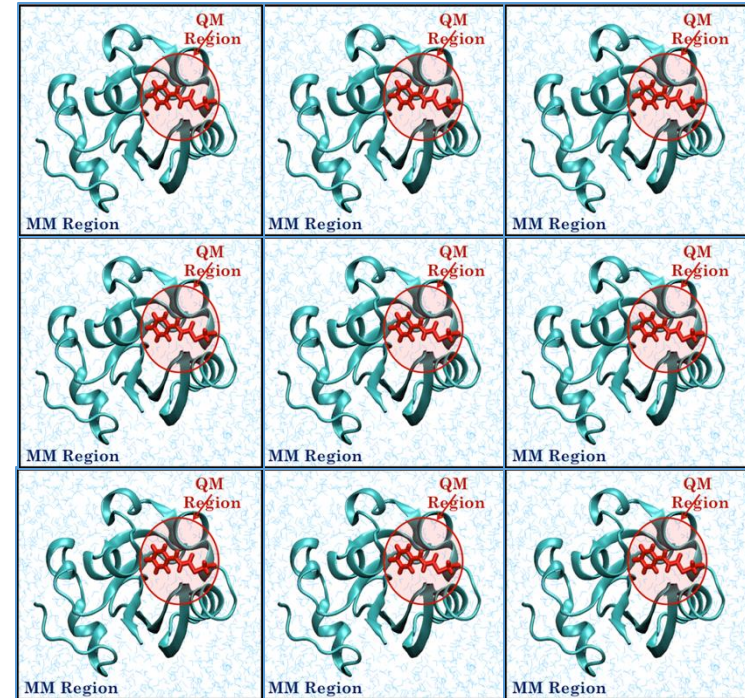
- Avoids issues with conventional Mulliken (or other) charge-based approaches
- Small error only in long-range QM/QM electrostatics (similar to robust RI / density fitting)
- Numerically stable with any basis set
- Efficient, performance impact ~ 10%

$$E_{\text{CEw}}[q] = \frac{1}{2} \langle q_{\text{mm}} | \widehat{J}_{\mathbf{n}} | q_{\text{mm}} \rangle + \frac{1}{2} \langle q_{\text{qm}} | \widehat{J} | q_{\text{qm}} \rangle \\ + \langle q_{\text{qm}} | \widehat{J}_{\mathbf{n}} | q_{\text{mm}} + q_{\text{qm}}^{\text{ref}} \rangle - \langle q_{\text{qm}} | \widehat{J} | q_{\text{qm}}^{\text{ref}} \rangle \\ - \frac{1}{2} \langle q_{\text{qm}}^{\text{ref}} | \widehat{J}_{\mathbf{n}} | q_{\text{qm}}^{\text{ref}} \rangle + \frac{1}{2} \langle q_{\text{qm}}^{\text{ref}} | \widehat{J} | q_{\text{qm}}^{\text{ref}} \rangle$$

$$\langle \mathbf{r} | \widehat{J} | q \rangle = \int \frac{q(\mathbf{r}')}{|\mathbf{r} - \mathbf{r}'|} d^3 r'$$

$$\langle \mathbf{r} | \widehat{J}_{\mathbf{n}} | q \rangle = \sum_{\mathbf{n}} \int \frac{q(\mathbf{r}' + \mathbf{n})}{|\mathbf{r} - \mathbf{r}'|} d^3 r'$$

Giese and York, *J. Chem. Theory Comput.* **12**, 2611 (2016)



$$\frac{1}{2} \langle q_{\text{qm}} - q_{\text{qm}}^{\text{ref}} | \widehat{J}_{\Delta} | q_{\text{qm}} - q_{\text{qm}}^{\text{ref}} \rangle \approx 0$$

$$\frac{1}{2} \langle q_{\text{qm}} | \widehat{J}_{\Delta} | q_{\text{qm}} \rangle \approx \langle q_{\text{qm}} | \widehat{J}_{\Delta} | q_{\text{qm}}^{\text{ref}} \rangle - \frac{1}{2} \langle q_{\text{qm}}^{\text{ref}} | \widehat{J}_{\Delta} | q_{\text{qm}}^{\text{ref}} \rangle$$

$$\langle \mathbf{r} | \widehat{J}_{\Delta} | q \rangle = \sum_{\mathbf{n} \neq 0} \int \frac{q(\mathbf{r}' + \mathbf{n})}{|\mathbf{r} - \mathbf{r}'|} d^3 r' = \langle \mathbf{r} | \widehat{J}_{\mathbf{n}} | q \rangle - \langle \mathbf{r} | \widehat{J} | q \rangle$$

QM/MM Ewald / PME long-range electrostatics – HF/DFT

Ambient Potential Composite Ewald Method

- Avoids issues with conventional Mulliken (or other) charge-based approaches
- Small error only in long-range QM/QM electrostatics (similar to robust RI / density fitting)
- Numerically stable with any basis set
- Efficient, performance impact ~ 10%

$$E_{\text{CEw}}[q] = \frac{1}{2} \langle q_{\text{mm}} | \hat{J}_{\text{n}} | q_{\text{mm}} \rangle + \frac{1}{2} \langle q_{\text{qm}} | \hat{J} | q_{\text{qm}} \rangle$$

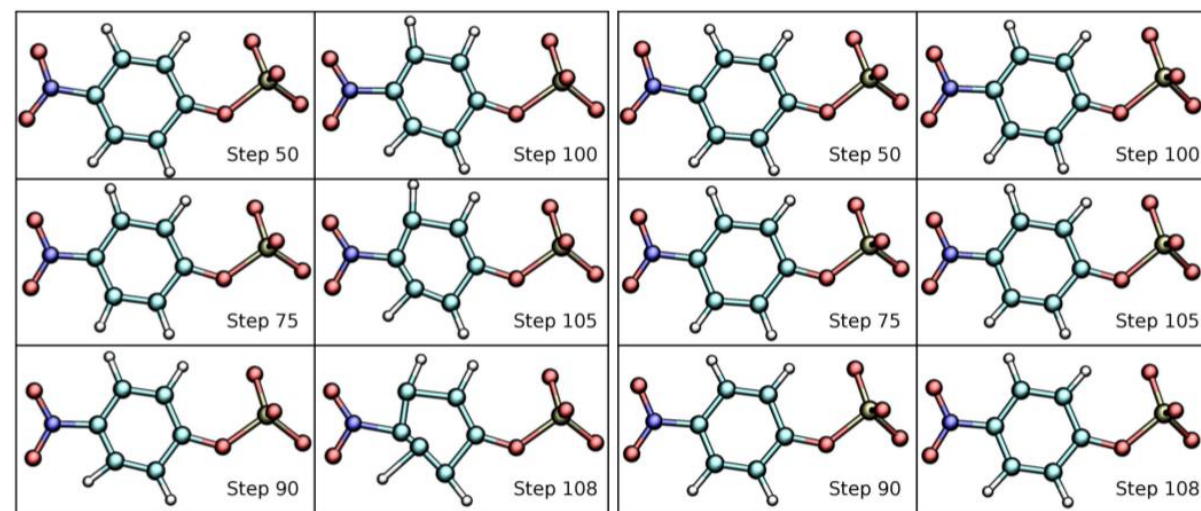
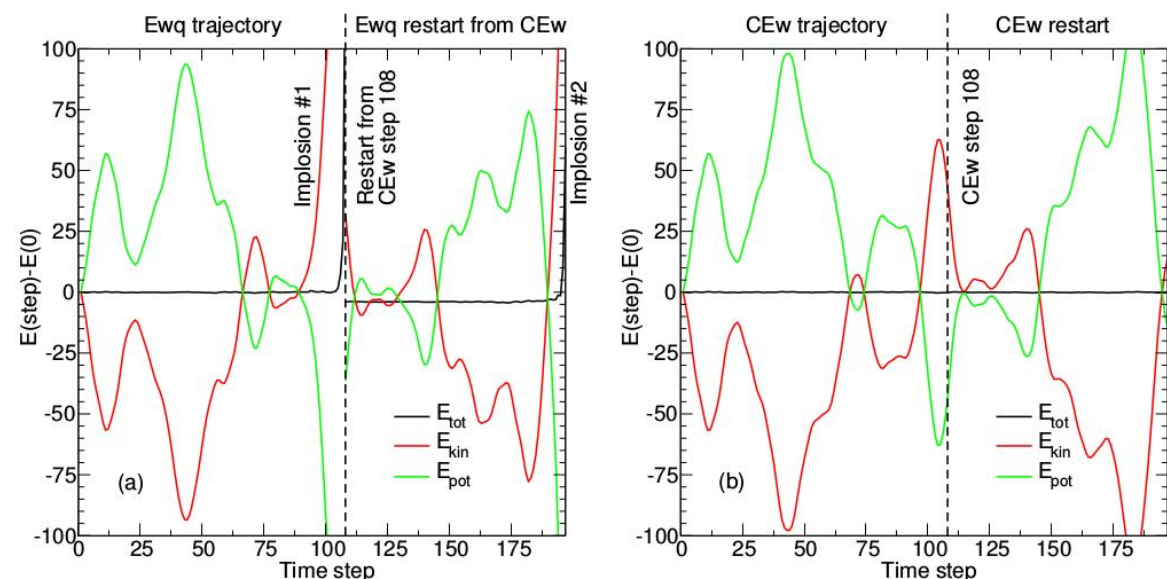
$$+ \langle q_{\text{qm}} | \hat{J}_{\text{n}} | q_{\text{mm}} + q_{\text{qm}}^{\text{ref}} \rangle - \langle q_{\text{qm}} | \hat{J} | q_{\text{qm}}^{\text{ref}} \rangle$$

$$- \frac{1}{2} \langle q_{\text{qm}}^{\text{ref}} | \hat{J}_{\text{n}} | q_{\text{qm}}^{\text{ref}} \rangle + \frac{1}{2} \langle q_{\text{qm}}^{\text{ref}} | \hat{J} | q_{\text{qm}}^{\text{ref}} \rangle$$

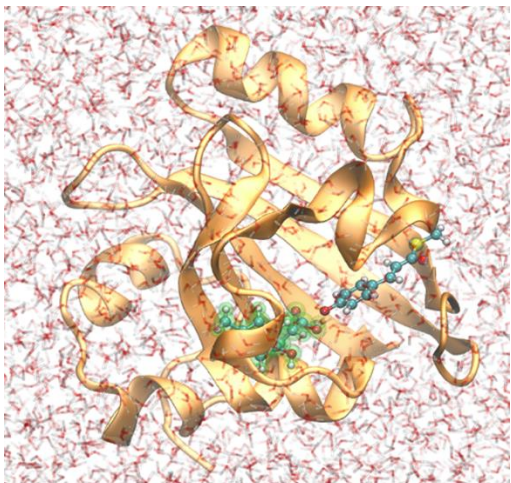
$$\langle \mathbf{r} | \hat{J} | q \rangle = \int \frac{q(\mathbf{r}')}{|\mathbf{r} - \mathbf{r}'|} d^3 r'$$

$$\langle \mathbf{r} | \hat{J}_{\text{n}} | q \rangle = \sum_{\mathbf{n}} \int \frac{q(\mathbf{r}' + \mathbf{n})}{|\mathbf{r} - \mathbf{r}'|} d^3 r'$$

Giese and York, *J. Chem. Theory Comput.* **12**, 2611 (2016)



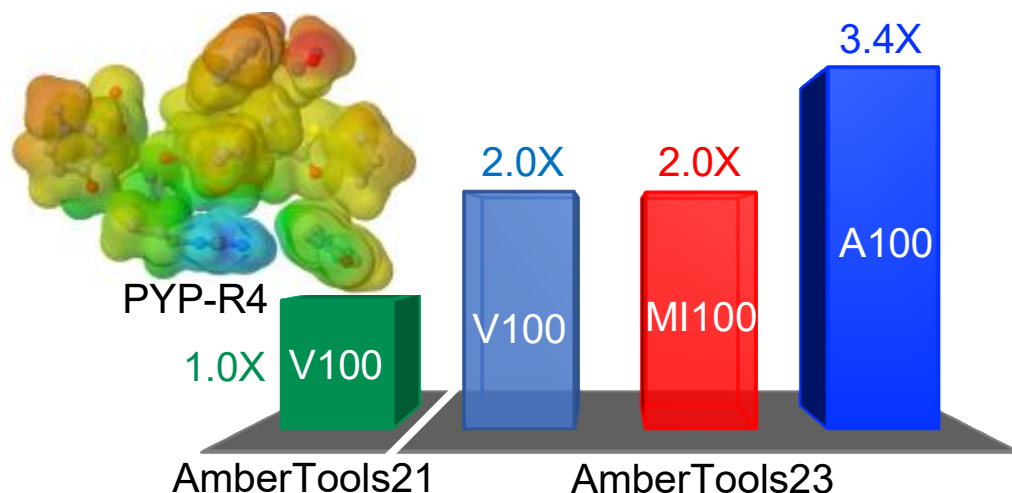
QUICK QM/MM Performance on Nvidia and AMD GPUs



Performance

- PYP (photoactive yellow protein) in water, PME
- Enables ps/day time-scale DFT-QM/MM MD

B3LYP/def2-SVP QM/MM



QM/MM performance in ps/day with 0.5 fs time step on Nvidia A100

# GPUs	6-31G*	def2-SVP
QM region = 49 atoms (440 / 509 basis functions)		
1	4.2	4.0
2	6.6	6.2
4	9.3	8.8
QM region = 159 atoms (1206 / 1479 basis functions)		
1	0.6	0.4
2	1.0	0.7
4	1.6	1.1

Manathunga, Aktulga, Götz, Merz, *J. Chem. Inf. Model.* **63** (2023), 711-717.

Amber free energy tools – Interoperable QM/MM/ML software

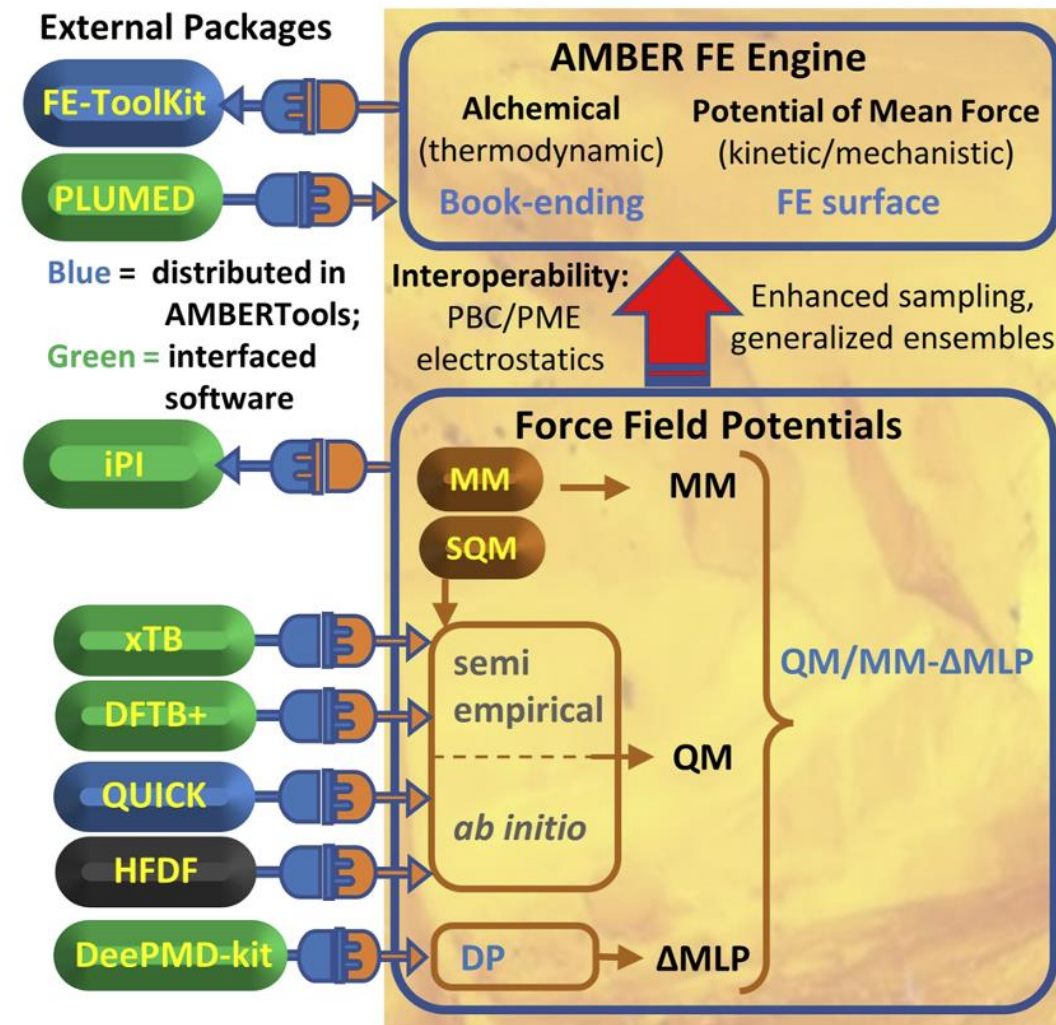
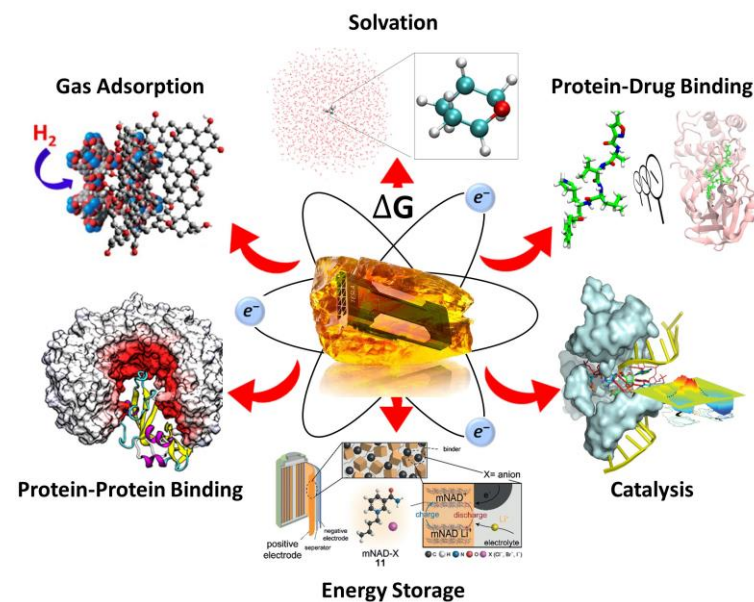
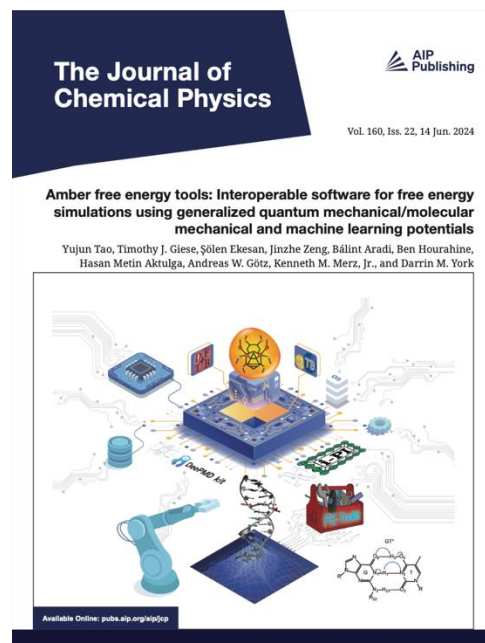
RESEARCH ARTICLE | JUNE 10 2024

Amber free energy tools: Interoperable software for free energy simulations using generalized quantum mechanical/molecular mechanical and machine learning potentials

FREE

Special Collection: **Modular and Interoperable Software for Chemical Physics**

Yujun Tao ; Timothy J. Giese ; Şölen Ekesan ; Jinzhe Zeng ; Bálint Aradi ; Ben Hourahine ; Hasan Metin Aktulga ; Andreas W. Götz ; Kenneth M. Merz, Jr. ; Darrin M. York 



Summary

The world of scientific computing is changing rapidly

- Current computers are heterogeneous mostly with CPUs and GPUs
- Different accelerators are emerging (FPGA, AI-ASICs, Quantum Computing, ...)
- Much of this is driven by tremendous advances in AI / Machine Learning
- Computational chemistry has to adapt and embrace new methods and new processors

QUICK: Open-source, multi-GPU capable quantum chemistry

- Available on Github at <https://github.com/merzlab/quick>
- HF and DFT with wide range of GGA and hybrid functionals

QUICK + Amber: Fast multi-GPU capable QM/MM code

- Free and open source, AmberTools available at <https://ambermd.org>
- User friendly integration with AMBER for QM/MM simulations

Acknowledgments

- **Vinicius Cruzeiro, Jamie Smith, Dmitry Pekurovsky, Etienne Palos, Vikrant Tripathy** (SDSC)
QUICK, QM/MM main developers, performance optimizations
- **Madu Manathunga, Akhil Shajan, Danil Kaliakin, Kurt O'Hearn**
Metin Aktulga, Kenneth Merz (MSU/CC)
QUICK, QM/MM main developers, performance optimizations
- **Dawei Mu** (NCSA)
Kristopher Keipert, Kamesh Arumugam, Jonathan Lefman, Stefan Maintz, Kyle Jacobs (Nvidia)
Gina Sitaraman, Mahdieh Ghazimirsaeed, Leopold Grinberg, Paul Bauer, Chip Freitag (AMD)
Performance optimizations
- **Tim Giese, Darrin York** (Rutgers)
Ambient Potential Composite Ewald method
- **Yipu Miao** (Meta)
Original GPU ERI implementation
- **Nvidia, AMD, Intel**
(hardware, software engineers)
- **NSF OAC-1835144, OAC-2209717, OAC-2209718, OAC-2435622**



<https://github.com/merzlab/quick>



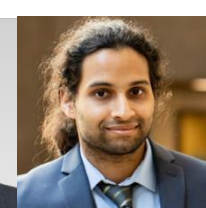
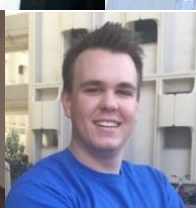
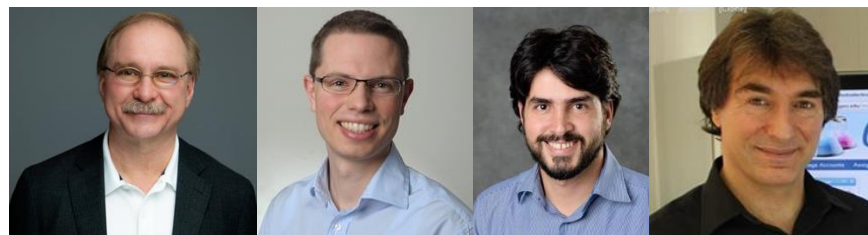
AMBER

<https://ambermd.org>



ACCESS

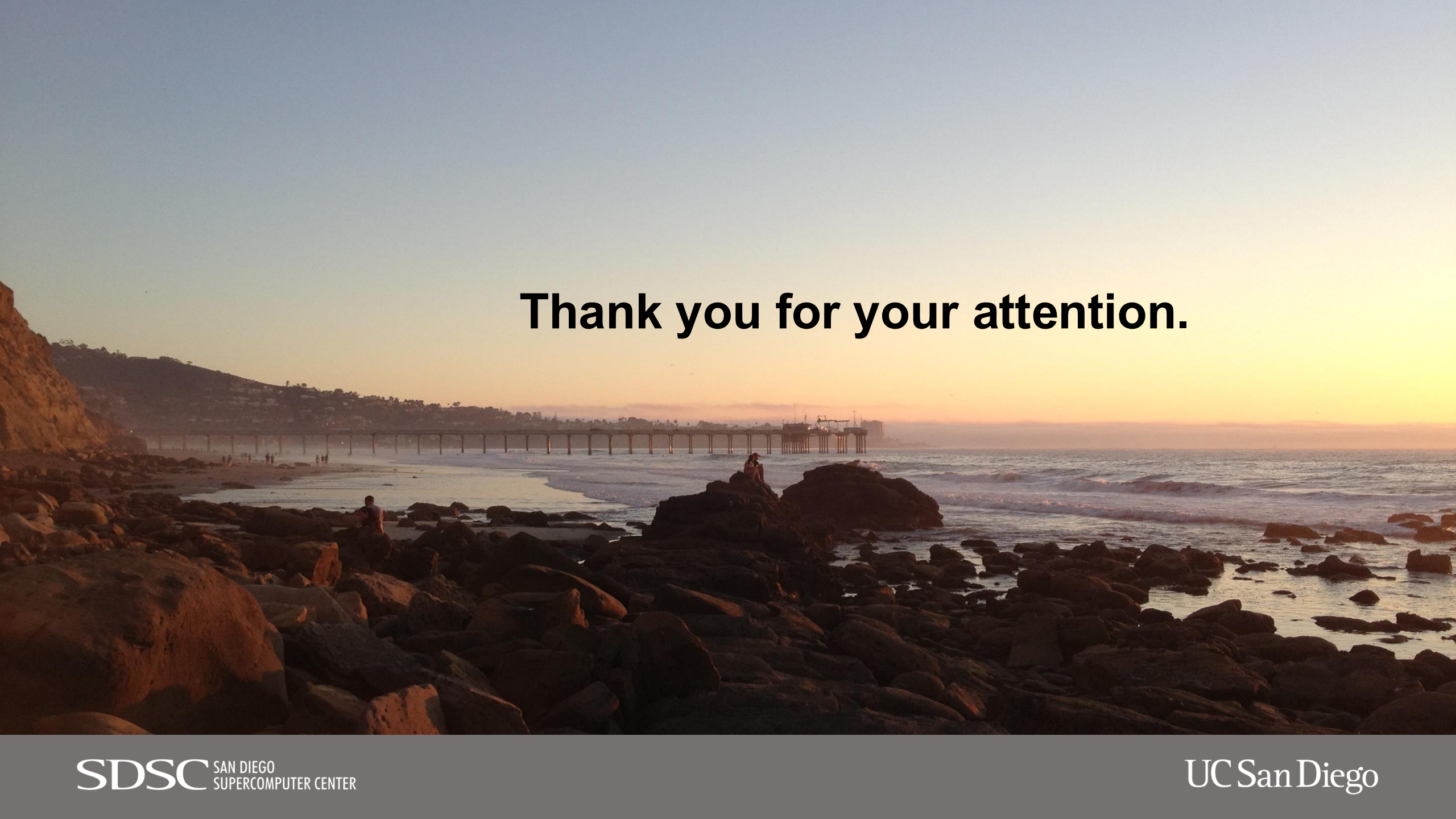
SDSC
SAN DIEGO SUPERCOMPUTER CENTER



Acknowledgments

- **Mahidhar Tatineni, Subhashini Sivagnanam, Igor Sfiligoi, Christopher Irving**
Cosmos PI team
- **Robert Sinkovits, Mary Thomas**
Applications, Education and Outreach
- **Yuwu Chen, Nicole Wolter, Fernando Silva, Trevor Cooper, Tom Hutton, Avery Blas**
Systems administration, user support, logistics
- **AMD, HPE, VAST Data**
(hardware, software engineers)
- **NSF OAC-2404323**





Thank you for your attention.